

REPUBLIQUE ALGERIENNE DEMOCRATIQUE ET POPULAIRE
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITE D'ORAN ES-SENIA



FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

MEMOIRE

Présenté par

Mme SAICHI Souad

Pour obtenir

LE DIPLOME DE MAGISTER

Spécialité Informatique

Option : Informatique et Automatique

Intitulé :

Optimisation de requêtes dans les entrepôts de données

Soutenu le 27 juin 2009 à la salle de conférences de la faculté des sciences

Devant les membres du jury:

Mr H. HAFFAF

Professeur, Université d'Oran, ES-Sénia, Algérie

(Président)

M. A. BENYETTOU

Professeur à l'USTO Mohamed Boudiaf, Oran, Algérie

(Examineur)

Melle F.BENDELLA

Maître de Conférences, l'USTO Mohamed Boudiaf, Oran, Algérie

(Examinatrice)

Mr B. ATMANI

Maître de Conférences, Université d'Oran, ES-Sénia, Algérie

(Examineur)

Mr B. BELDJILALI

Professeur, Université d'Oran, ES-Sénia, Algérie

(Rapporteur)

Mr L. BELLATRECHE

Maître de Conférences, Université de Poitiers, France

(Invité)

Résumé

La fragmentation de données est une des techniques utilisée dans la conception physique des entrepôts de données, elle permet d'accélérer l'exécution des requêtes et de faciliter la gestion des données de l'entrepôt. La meilleure manière de fragmenter un entrepôt de données relationnel consiste d'abord à décomposer les tables de dimension ensuite à utiliser des schémas de fragmentation pour partitionner la table de faits. L'espace de recherche pour sélectionner le schéma de fragmentation optimal peut être très important. Nous proposons de formaliser d'abord le problème de sélection d'un schéma de fragmentation pour un entrepôt de données relationnel comme problème d'optimisation avec une contrainte de maintenance.

Nous proposons ensuite une méthode hybride combinant un algorithme tabou et un algorithme de séparation évaluation pour résoudre ce problème

Mots-clés

Entrepôt de données, Fragmentation, Schéma optimal, Algorithme Tabou, Algorithme de séparation/évaluation.

Abstract

The fragmentation of data is one of the techniques used in the physical design of data warehouses, it helps accelerate the execution of requests and facilitate management of data warehouse. The best way to fragment a relational data warehouse is first to break down tables dimension then use patterns of fragmentation to partition the table of facts. The space research to select the optimal pattern of fragmentation can be very important. We propose to formalize the first problem of selecting a pattern of fragmentation for a relational data warehouse as optimization problem with constraint maintenance.

We then offer a hybrid approach combining an algorithm taboo and a separate assessment algorithm to solve this problem

Key words

Data warehouse, Fragmentation, optimal Diagram, Algorithm Taboo, Algorithm of separation/evaluation.

Remerciements

Cette thèse, bien que signée de mon seul nom, ne doit donc pas être attribuée à un travail solitaire : elle reflète ces années de travail mené ensemble ; de jour, de nuit, de week-end, de jours fériés... Je tiens à remercier ici tous ceux qui m'ont aidé, soutenu et encouragé pendant ma thèse.

Mes premiers remerciements vont bien entendu à mon jury. Je tiens tout d'abord à remercier Monsieur HAFFAF HAFID pour m'avoir fait l'honneur de présider mon jury. Je remercie également chaleureusement Mademoiselle BENDELLA FATIMA Monsieur BENYETTOU ABDELKADER et Monsieur ATMANI BAGHDAD, tous rapporteurs, qui ont consacré une partie de leur temps précieux à relire ce manuscrit et à faire des commentaires constructifs. Et évidemment, n'oublions pas mes deux encadreurs.

M. BOUZIANE BELDJILALI et M. LADJEL .BELLATRECHE qui m'ont fait confiance pendant ces années, je tiens à remercier MEKKAKIA, BOUDIA, DERKAOUI, BENGUEDDACH, et ROUBA.

Merci aussi à tous les autres que j'oublie de citer ici et qui ont contribué d'une façon ou d'une autre à cette thèse, comme mes amis pour les moments inoubliables qu'on a passé ensemble.

Je remercie mon défunt père qui était un homme d'honneur et qui m'a toujours poussé vers l'avant pour mes études.

Je tiens évidemment à remercier ma mère, mes frères et mes sœurs, pour ce qu'ils sont et parce que rien ne serait si bien sans eux. Merci à mon mari SID AHMED, pour qui, chaque jour, je fais de mon mieux pour être à ses yeux une véritable héroïne.

Enfin, merci à ceux qui ont su me donner l'envie, la joie et la soif d'évoluer. Mes deux enfants AHMED RACHID et AMINA.

Sommaire

RESUME	7
MOTS-CLES.....	7
ABSTRACT	7
REMERCIEMENTS	6

Chapitre 1

1. INTRODUCTION	8
2. LES ENTREPOTS DE DONNEES	8
2.1 DEFINITIONS.....	8
2.2 LES CARACTERISTIQUES DE DONNEES D'ENTREPOTS.....	9
2.3 L'EXPLOITATION D'UN ENTREPOT DE DONNEES	10
2.4 CONCEPTION D'UN ENTREPOT DE DONNEES	11
2.5 LES MODELES ET LES LANGAGES DE MODELISATION.....	11
2.5.1 Schéma en étoile.....	11
2.5.2 Schéma en flocon de neige	12
2.5.3 Schéma en constellation de faits	13
2.6 ARCHITECTURE D'UN ENTREPOT DE DONNEES	13
2.6.1 Architecture centralisée (Corporated architecture).....	14
2.6.2 ARCHITECTURE FEDEREE (FEDERATED ARCHITECTURE).....	15
2.6.3. Architecture trois-tiers (Three-tiers architecture).....	15
3 PROBLEMATIQUE	16
4 TECHNIQUES D'OPTIMISATION.....	16
4.1 LES VUES MATERIALISEES	17
4.2 LES INDEX	18
4.2.1 Techniques d'indexation	19
4.2.2 Sélection d'index	22
4.3 LA FRAGMENTATION	24
4.3.1 La fragmentation verticale	24
4.3.2 La fragmentation horizontale.....	25
4.3.3 La fragmentation mixte	27
4.3.4 Évolution de la fragmentation dans les SGBD commerciaux.....	28
5 CONCLUSION	28

Chapitre 2

1 INTRODUCTION	30
2 METHODOLOGIE DE FRAGMENTATION HORIZONTALE DANS LES ENTREPOTS DE DONNEES	30
2.1 PROCESSUS DE GENERATION DE SCHEMA	34
2.2 REPRESENTATION DES FRAGMENTS HORIZONTAUX.....	34
2.3 IDENTIFICATION DES FRAGMENTS PARTICIPANTS A UNE REQUETE	35
3 MODELE DE COUT.....	36
3.1 COMPOSANTES D'UN MODELE DE COUT	36
3.2 STATISTIQUES ET ESTIMATIONS	37

4 CONCLUSION	38
--------------------	----

Chapitre 3

1 INTRODUCTION	39
2 ALGORITHME TABOU	39
3 ALGORITHME SEPARATION / ÉVALUATION	42
4 MISE EN ŒUVRE DE LA DEMARCHE	42
4.1 LE GENERATEUR DE SCHEMAS	43
4.2 LE MODELE DE COUT DANS NOTRE CONTEXTE	46
4.2.1 <i>Les hypothèses</i>	46
4.2.3 <i>La formule du modèle de coût</i>	47
4.3 ALGORITHME PROPOSE	47
5 SCENARIO EXPERIMENTE	49
6 DISCUSSION DES RESULTATS	53
7 CONCLUSION	56
BIBLIOGRAPHIE	82

Liste des Figures

Figure 1 Schéma en étoile (star schema).....	12
Figure 2 Schéma en flocon de neige	12
Figure 3 Schéma en constellation.....	13
Figure 4 Architecture conceptuelle d'un entrepôt de données	13
Figure 5 Architecture centralisée	15
Figure 6 Architecture fédérée.....	15
Figure 7 Architecture trois-tiers	16
Figure 8 Techniques d'optimisation.....	16
Figure 9 Index en B-arbre construit sur l'attribut <code>Personne_Nom</code>	19
Figure 10 Index de hachage construit sur l'attribut <code>Nom</code>	20
Figure 11 Index bitmap construit sur le sexe des clients.....	21
Figure 12 L'architecture de l'outil de sélection d'index	22
Figure 13 Fragmentation verticale	24
Figure 14 Fragmentation Horizontale	25
Figure 15 Fragmentation mixte	27
Figure 16 Organigramme de l'application	43
Figure 17 Schéma en étoile de l'entrepôt.....	49
Figure 18 Les étapes de notre algorithme proposé.....	52
Figure 19 Nombre d'E/S par rapport au nombre d'attributs utilisées	54
Figure 20 Effet du seuil <code>W</code>	55
Figure 21 Temps d'exécution de chaque algorithme	55

Liste des Tableaux

Tableau 1 La table de spécification de la fragmentation.....	35
Tableau 2 les six prédicats	45
Tableau 3 L'ensemble des prédicats et les tables de dimension correspondantes	50
Tableau 4 les fragments des tables de dimension	51
Tableau 5 Les fragments de la table des faits.....	52

Glossaire

Jointure : En gestion de base de données, une jointure est un lien combinant les enregistrements de deux tables disposant de valeurs correspondantes dans un champ commun.

Méta données : Une méta donnée est une «donnée sur des données».

MOLAP : Multidimensional On-Line Analytical Processing.

OLAP : OnLine Analytical Processing. Architecture de programme où l'aspect décisionnel en temps réel est mis en avant.

ROLAP : Relational OLAP. Analyse complexe de données, analyse de données multidimensionnelle efficace. Permet un travail avec des objets d'analyse sans connaissance nécessaire sur les structures de données et un accès facile aux données.

Schéma de Fragmentation : Un schéma de fragmentation est le résultat du processus de fragmentation d'une table donnée

Sélectivité : est un coefficient représentant le nombre d'objets sélectionnés rapporté à un nombre d'objets total d'une table elle varie entre 0 et 1.

Table De Faits : Un ensemble de données du même type, permettant de structurer la base multidimensionnelle. Une dimension est parfois appelée un axe. Chaque cellule d'une mesure est associée à une seule position de chaque dimension. Temps, pays, produit sont des dimensions classiques.

Vues Matérialisées calculent à l'avance des résultats de requêtes SQL dans une base de données et les conservent physiquement pour accélérer les traitements.

Introduction Générale

La technologie des entrepôts de données (data warehouses, dans la terminologie anglo-saxonne) et de l'analyse multidimensionnelle en ligne OLAP (On-Line Analytical Processing) développe des outils décisionnels qui permettent d'étudier, par exemple, le comportement de consommateurs, de produits, de sociétés ; d'effectuer une veille concurrentielle ou technologique, etc. Pour cela, ils intègrent traditionnellement des données dites de production dans une base de données centralisée à vocation décisionnelle (l'entrepôt), où elles sont agrégées, historisées et structurées de manière à en permettre et à en optimiser l'analyse en ligne.

La fragmentation est une technique de conception logique introduite dans les bases de données réparties. La fragmentation consiste à partitionner une table horizontalement ou verticalement de façon à réduire le nombre des accès nécessaires pour le traitement de certaines requêtes. Dans notre étude, nous nous intéressons à la fragmentation horizontale qui semble être une réponse au problème de réduction du temps d'exécution des requêtes décisionnelles. En effet, elle a été introduite dans les bases de données réparties dans le but de minimiser le nombre d'entrées-sorties (ou le coût de transfert de données) pendant l'exécution des requêtes.

L'objectif visé par notre étude consiste à fournir un schéma de fragmentation optimal qui permet d'optimiser les performances des requêtes. Cette technique d'optimisation repose sur des méthodes de fragmentation. Nous proposons un modèle de coût pour évaluer le coût d'exécution d'un ensemble de requêtes sur un schéma en étoile fragmenté. Durant le processus de fragmentation, nous avons remarqué que le choix du schéma de fragmentation optimal influe sur le coût d'exécution des requêtes. L'algorithme proposé «Tabou combiné avec séparation/évaluation » a pour but la sélection du «meilleur» schéma.

A cet effet notre mémoire est organisé comme suit :

Le premier chapitre s’articule autour des entrepôts de données portant sur les différents types de données manipulées, leurs organisations dans une base de données et dans les entrepôts. Ensuite, les objectifs pour une conception d’un entrepôt de données ainsi que les modèles et les langages de modélisation. Enfin, les différentes architectures des entrepôts.

Le deuxième chapitre expose les techniques d’optimisation des requêtes, à savoir les vues matérialisées, les index et la fragmentation ainsi que les modèles du coût. .

Le Troisième chapitre présente notre démarche de conception pour la résolution du problème énoncé. Nous exposons la démarche à suivre et nous détaillons le mode de fonctionnement de chacune des étapes de manière progressive. Nous décrivons notre algorithme proposé pour la sélection de schéma optimal. Enfin la phase d’expérimentation synthétise les résultats qui s’avèrent prometteuses.

En conclusion, nous établissons un bilan de nos travaux ainsi que d’éventuelles perspectives.

Chapitre 1

*ENTREPÔTS DE DONNÉES ET TECHNIQUES
D'OPTIMISATION*

1. Introduction

Actuellement, les données utilisées et échangées par les applications décisionnelles sont de plus en plus diverses et hétérogènes.

La technologie des entrepôts de données (DataWarehouses) et de l'analyse multidimensionnelle on line OLAP (On Line Analytical Processing) développe des outils décisionnels qui permettent d'étudier, par exemple, le comportement de consommateurs, de produits, de sociétés; d'effectuer une veille concurrentielle ou technologique, etc. pour cela, on intègre traditionnellement des données dites de production dans une base de données centralisées à vocation décisionnelle qu'on appelle entrepôt, où elles sont agrégées, historisées et structurées de manière à en permettre et à en optimiser l'analyse en ligne.

2. Les entrepôts de données

2.1 Définitions

Il existe plusieurs définitions d'un entrepôt de données (Data warehouse), selon certains auteurs [IWH94], [INM97], [TDB00]:

Définition 1: Les entrepôts de données sont définis par Inmon et Hackarton [IWH94] comme « une collection de données orientées sujet, intégrées, historisées et persistantes, utilisée pour le support d'un processus d'aide à la décision. »

Définition 2: Un entrepôt de données doit être organisé autour des sujets de l'entreprise (clients, étudiants, produits, etc.) [INM97]. L'entrepôt doit aussi être intégré, c'est-à-dire donner une définition constante de tous les termes et des données qu'il contient. Le vocabulaire utilisé dans l'entrepôt doit être le même, peu importe la personne qui l'utilise. Les données ont une période de validité dans le temps, il est possible de déterminer avec précision quand chaque enregistrement a été inséré dans l'entrepôt. Il est recommandé de ne pas écraser les anciens enregistrements, ce qui permet de recréer un portrait de l'entreprise dans le temps. L'ensemble de l'entrepôt doit être conçu pour faciliter l'accès aux utilisateurs finaux avec des logiciels d'analyse de données. Ces logiciels sont généralement conçus pour permettre aux décideurs de prendre des décisions plus éclairées en leur donnant accès aux données rapidement et facilement, d'où le terme business intelligence.

Définition 3: Un entrepôt de données peut être vu comme « un ensemble de vues matérialisées définies par des relations sur des sources de données distantes » [TDB00]. Cette définition semble

être une simple explication d'une méthode pratique pour réaliser un entrepôt, les vues matérialisées ne permettent pas de résoudre tous les problèmes d'implémentation d'un entrepôt, même si elles peuvent faciliter le chargement des données. Cette définition ne tient pas compte de la nature historique d'un entrepôt, elle ne prévoit pas de méthode pour historiser les données qui proviennent des sources de données de l'entrepôt. Des tables supplémentaires sont nécessaires pour créer un historique, car une vue matérialisée effectue une copie des données et supprime la version précédente.

L'entrepôt de données est destiné à fournir de l'information :

- ☒ Thématique, c'est à dire relative à un domaine intéressant le décideur possédant une référence temporelle,
- ☒ Sûre, c'est à dire dont la qualité a été vérifiée selon [LHE95] et [BRI00],
- ☒ Facile d'accès,
- ☒ Non volatile, car régulièrement complétée et rarement «nettoyée».

Ce que l'on demande aux outils actuels c'est de permettre une extraction fiable des données du système d'information pour construire le système d'information stratégique et, aussi bien sûr, des possibilités d'exploitation bien meilleures qu'avec les environnements informatiques existants. Il existe différents types de données manipulées par l'entrepôt :

J.-M. Franco [FR97b] détaille et complète les notions abordées par la définition de [IWH94] sur les données.

2.2 Les caractéristiques de données d'entrepôts

Détaillées : issues des bases de données de production. Elles reflètent les événements les plus récents. Des intégrations régulières de données issues des systèmes de production sont réalisées à ce niveau.

Orientées sujet : les données sont organisées par thèmes et non pas par processus fonctionnels, comme c'est l'habitude dans les organisations traditionnelles. L'intérêt est de disposer de l'ensemble des informations sur un sujet le plus souvent transversal aux structures fonctionnelles de l'entreprise. Cette approche permet également de développer le système décisionnel via une démarche incrémentale sujet après sujet.

Intégrées : afin d'assurer la présentation de données homogènes, celles-ci doivent être mises en

forme et unifiées afin d'avoir un état cohérent. Une donnée doit avoir une description et un codage uniques. Cette phase d'unification, qui d'apparence est simple, est en réalité complexe du fait de l'hétérogénéité des bases de données.

Historisées : dans un système de production, la donnée est sans cesse mise à jour à chaque nouvelle transaction. L'ancienne valeur est perdue. Ces systèmes conservent assez rarement un historique des données. Dans un entrepôt de données, la donnée ne doit jamais être mise à jour car elle représente une valeur insérée à un certain moment. Cette démarche induit la gestion d'un référentiel de temps associé à la donnée pour l'identification de cette donnée.

Non volatiles : c'est une conséquence de l'historisation décrite ci-dessus.

Agrégées : ce sont des résultats et des synthèses d'analyse, accessibles à tous, et correspondant à des éléments d'analyse représentatifs des besoins utilisateurs. Elles constituent déjà un résultat d'analyse et une synthèse de l'information contenue dans le système décisionnel, et doivent être facilement accessibles et compréhensibles.

De plus un datamart représente un magasin de données. Il s'agit d'une solution départementale d'entrepôt de données supportant une partie des données et fonctions de l'entreprise (produit, département, activité, etc.). C'est un sous ensemble d'entrepôt qui ne contient que les données d'un métier de l'entreprise alors que l'entrepôt contient toutes les données décisionnelles de l'entreprise pour tous les métiers.

2.3 L'exploitation d'un entrepôt de données

Puisqu'un entrepôt de données est différent d'une base de données traditionnelle, des logiciels différents sont nécessaires pour l'exploiter. Les logiciels OLAP utilisent une structure de données basée sur le modèle dimensionnel. À partir d'une ou plusieurs tables de faits et de plusieurs tables représentant des dimensions, l'utilisateur est capable de combiner les données à différents niveaux d'agrégation pour trouver des informations. OLAP appliqué à un entrepôt permet de parcourir une très grande quantité de données beaucoup plus rapidement que ce qui était possible auparavant. De plus, selon les besoins des utilisateurs, il est possible de prévoir des calculs d'agrégation durant le chargement des données dans l'entrepôt, ce qui permet d'avoir des temps de réponse beaucoup plus intéressants avec les différents algorithmes utilisés.

2.4 Conception d'un entrepôt de données

Plusieurs éléments tels que l'infrastructure système, les méta données, la découverte des données, l'acquisition des données, la distribution des données et les logiciels d'analyse [ONB97] (voir Annexe 3) doivent être considérés quand on veut créer un entrepôt. Un autre élément à considérer est la structure que l'on veut utiliser pour conserver les données. La mise en œuvre de ces éléments peut prendre beaucoup de temps. La conception d'un entrepôt n'est pas un exercice simple. Un entrepôt de données peut prendre plusieurs années et des millions de dollars à concevoir dans une grande entreprise, et nécessite la mise en place d'une bonne équipe de développement [THE 98].

2.5 Les modèles et les langages de modélisation

Selon le rôle que l'entrepôt est appelé à jouer dans l'entreprise, plusieurs modèles pour les données peuvent être proposés. Les modèles au cœur de la recherche sur les entrepôts de données sont : le modèle dimensionnel et des extensions du modèle entité relation standard [VSS02], le modèle choisi pour l'entrepôt peut être représenté par le langage UML (Unified Modeling Language). Le modèle le plus souvent recommandé est le modèle dimensionnel, avec le schéma en étoile [ONB97] et [MHP99].

2.5.1 Schéma en étoile

Ce modèle fonctionne avec une table de faits, c'est le centre du schéma. Le schéma en étoile représente une table de faits connectée à un ensemble de tables de dimensions. Chaque enregistrement dans la table de faits constitue un fait, (l'unité de base). La granularité du schéma permet de déterminer ce qui sera un fait. Ce modèle est recommandé à cause de sa faible complexité, sa facilité de compréhension pour l'utilisateur final et pour les liens directs avec les structures logiques des données [VSS02].

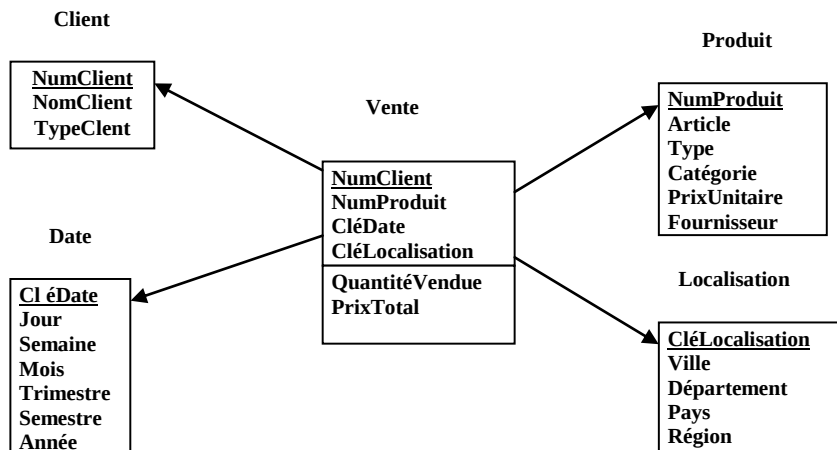


Figure 1 Schéma en étoile (star schema)

Dans la figure 1, on voit que la table de faits est Vente, et que les tables de dimensions sont Client, Produit, Date et Localisation. Ces dernières sont toutes liées par une clé à la table Ventes.

2.5.2 Schéma en flocon de neige

Ce modèle est une sorte de compromis entre les modèles relationnels et dimensionnels. Le schéma en flocon est supposé diminuer la redondance du schéma en étoile en normalisant certaines des tables de dimensions, surtout lorsqu'elles contiennent beaucoup d'enregistrements. (Un raffinement du schéma en étoile où certaines hiérarchies de dimensions sont normalisées en un ensemble de tables de dimensions plus petites) [FRA98].

Cependant, il ne faut pas transformer les dimensions en flocons, même quand elles sont grandes, cela entraîne de mauvaises performances de navigation [KIM96].

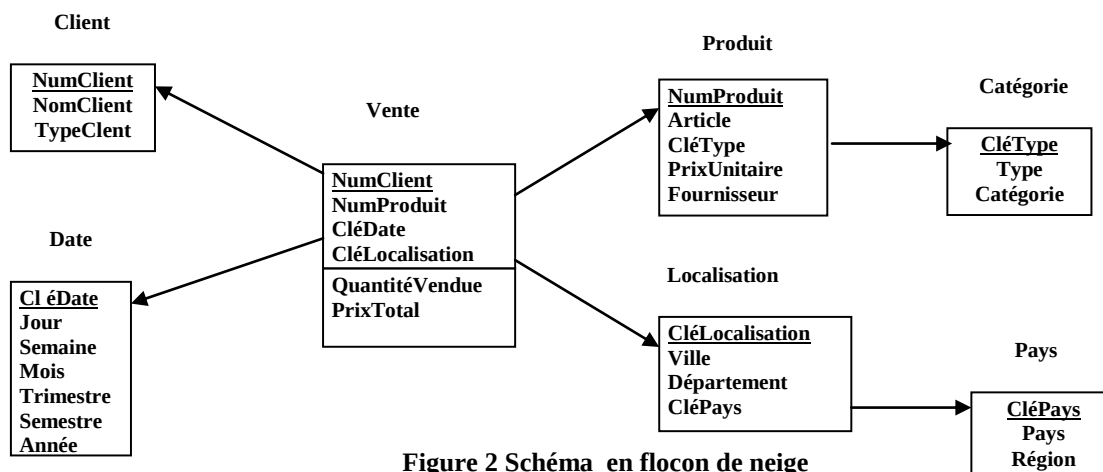


Figure 2 Schéma en flocon de neige

2.5.3 Schéma en constellation de faits

Plusieurs tables de faits partagent des tables de dimensions. Peut-être vu comme une collection d'étoiles (schéma en galaxie ou constellation de faits). Plusieurs tables de faits partageant quelques tables de dimension

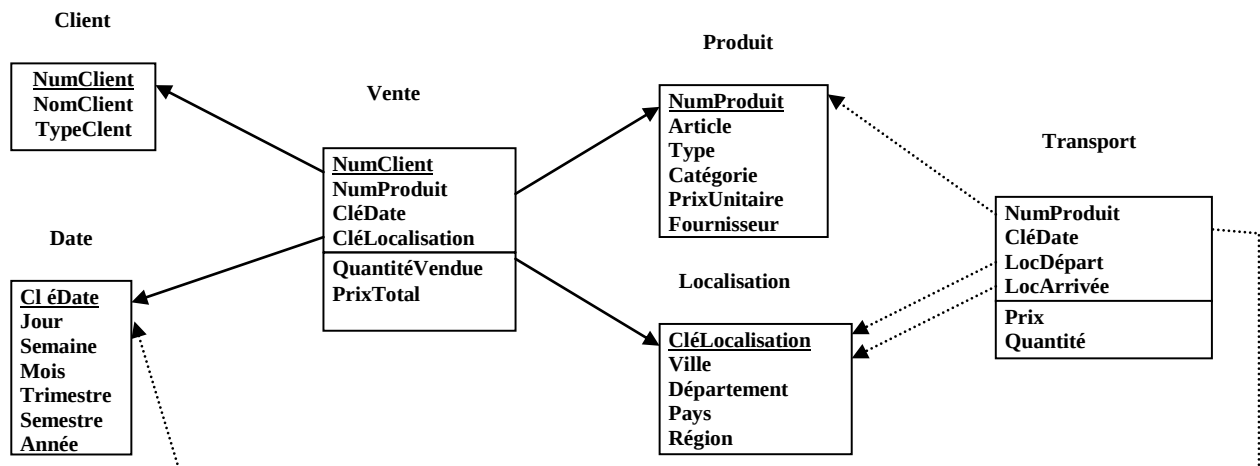


Figure 3 Schéma en constellation

2.6 Architecture d'un entrepôt de données

L'architecture d'un entrepôt de données peut être illustrée selon le schéma ci-dessous :

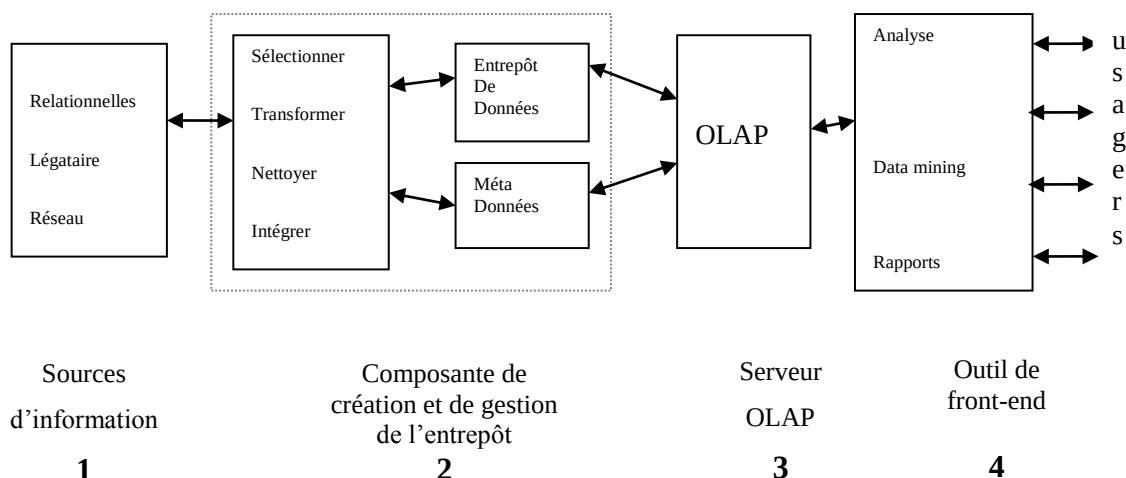


Figure 4 Architecture conceptuelle d'un entrepôt de données

Avant d'être chargées dans l'entrepôt, les données sélectionnées doivent être extraites des sources (1) et soigneusement épurées, pour éliminer les erreurs et réconcilier les différences sémantiques

(Nettoyage). Une fois ces données nettoyées, elles seront intégrées dans l'entrepôt (2), qui contient des données détaillées, des vues matérialisées et des données multidimensionnelles. Lors de changements dans des données sources, ces dernières sont propagées vers l'entrepôt (Rafraîchissement) [WB97].

Les méta-données contiennent des informations concernant la création, la gestion, et l'usage de l'entrepôt. Les méta-données sont stockées dans un répertoire différent de celui de l'entrepôt.

Elles sont considérées comme un pont entre l'utilisateur et l'entrepôt. L'entrepôt est accédé par un serveur OLAP (3) afin de présenter les sous formes multidimensionnelles aux clients pour des besoins informationnels (datamining, rapport,...). Le serveur OLAP interprète les requêtes des clients et les convertit en requête d'accès à l'entrepôt ou aux sources opérationnelles. Finalement, Le serveur OLAP fournit des vues multidimensionnelles des données aux outils de front-end (4), et ces derniers formatent les données conformément aux besoins des usagers.

Une architecture d'entrepôt de données exige ce qui suit :

- ☒ les données sources sont extraites de systèmes, de bases de données et de fichiers
- ☒ les données sources sont nettoyées, transformées et intégrées avant d'être stockées dans l'entrepôt
- ☒ l'entrepôt est en lecture seulement et est défini spécifiquement pour la prise de décision organisationnelle
- ☒ les usagers accèdent à l'entrepôt à partir d'interfaces et d'applications (clients)

Il existe d'autres architectures d'un entrepôt de données ;

2.6.1 Architecture centralisée (Corporated architecture)

Il s'agit de la version centralisée et intégrée d'un entrepôt regroupant l'ensemble des données de l'entreprise. Les différentes bases de données sources sont intégrées et sont distribuées à partir de la même plate-forme physique.

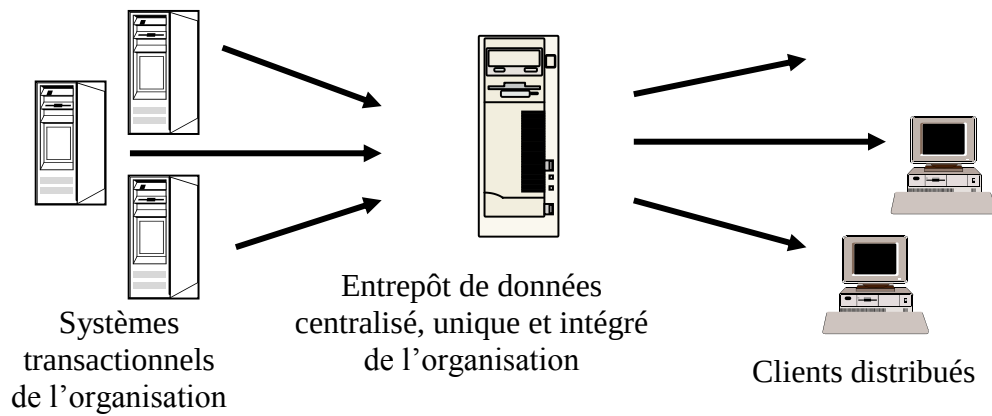


Figure 5 Architecture centralisée

2.6.2 Architecture fédérée (Federated architecture)

Il s'agit de la version intégrée d'un entrepôt où les données sont introduites dans les marchés de données orientés selon les différentes fonctions de l'entreprise.

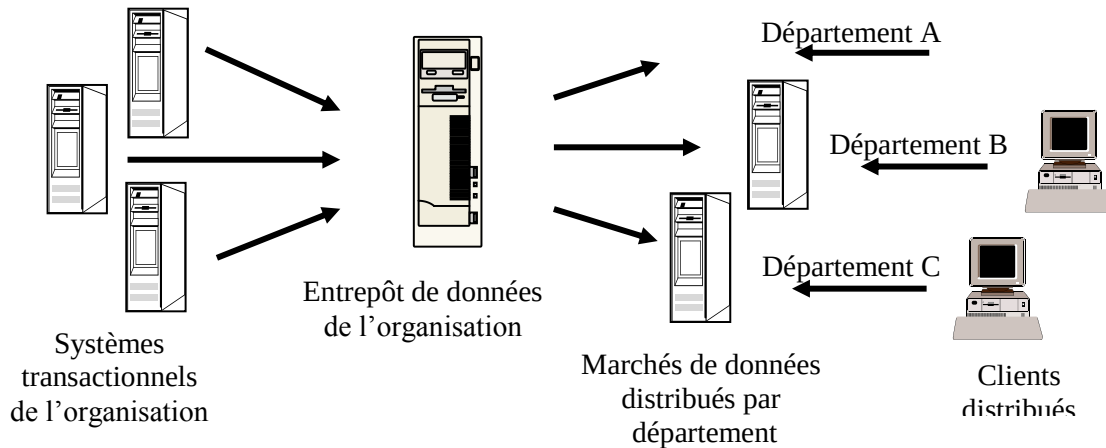


Figure 6 Architecture fédérée

2.6.3. Architecture trois-tiers (Three-tiers architecture)

Il s'agit d'une variante de l'architecture fédérée où les données sont divisées par niveau de détails.

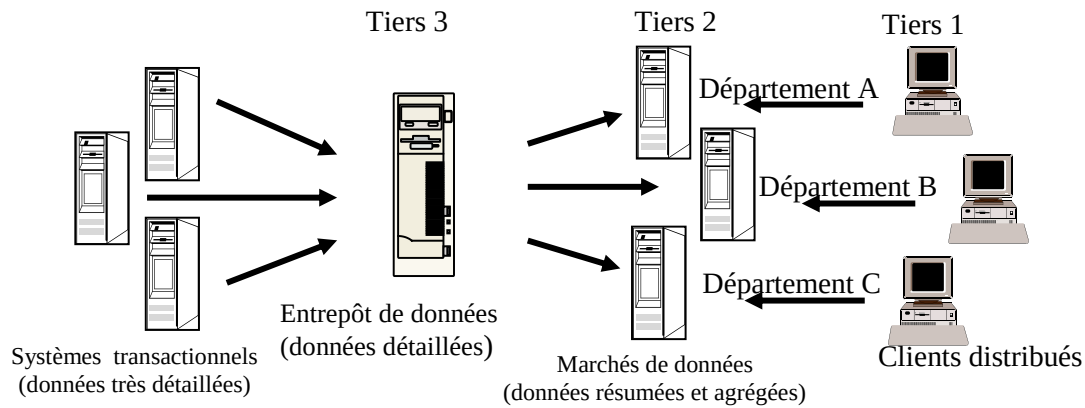


Figure 7 Architecture trois-tiers

3 Problématique

Vu la taille importante des données dans un entrepôt de données, qui rend leur interrogation lente mesurée par le temps de réponse d'une part, et d'autre part la complexité des requêtes décisionnelles qu'il l'exploite. Cette complexité est due aux opérations de jointure et d'agrégation utilisées par les requêtes, qui détériorent de manière significative les performances de l'entrepôt.

De ce fait, il apparaît donc nécessaire de concevoir des techniques pour l'optimisation des performances des requêtes d'entrepôts de données.

4 Techniques d'optimisation

Pour remédier au problème énoncé dans la problématique, plusieurs travaux ont vu le jour. Nous présentons trois solutions d'optimisation, à savoir les vues matérialisées, les index et la fragmentation illustrés dans la figure ci après.

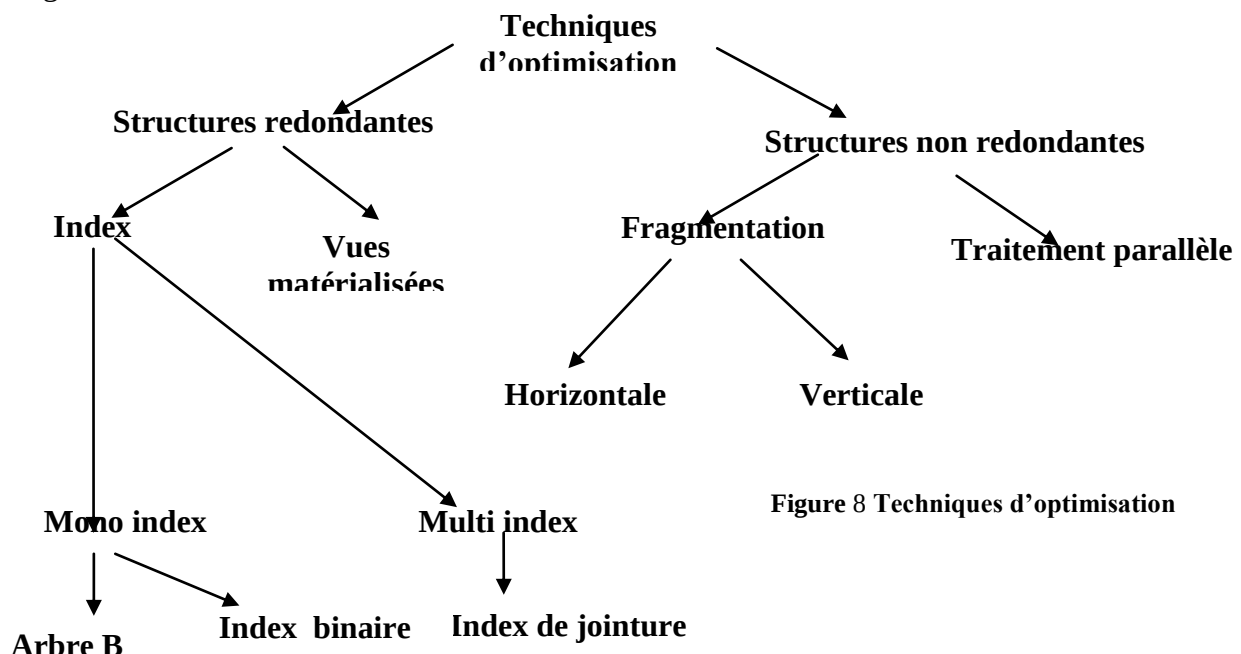


Figure 8 Techniques d'optimisation

4.1 Les vues matérialisées

Une vue matérialisée est une table contenant les résultats d'une requête. Les vues améliorent l'exécution des requêtes en pré calculant les opérations les plus coûteuses comme les jointures et les agrégations, en stockant leurs résultats dans la base. En conséquence, certaines requêtes nécessitent seulement l'accès aux vues matérialisées et sont ainsi exécutées plus rapidement. Cependant, la mise à jour des données implique systématiquement celle des vues matérialisées calculées à partir de ces données afin de conserver la cohérence et l'intégrité des données. Cela induit une surcharge du système liée au coût de maintenance des vues matérialisées. De plus, la matérialisation des vues requiert un espace de stockage additionnel que l'administrateur alloue à ces vues.

Problème de sélection de vue matérialisée: Il s'agit de déterminer l'ensemble de vues à matérialiser en tenant compte d'un certain nombre de paramètres comme les requêtes les plus fréquentes, l'espace de stockage et le coût de maintenance

Problème de maintenance de vue matérialisée : le coût d'exécution des requêtes est en conflit avec le coût de maintenance des vues car la matérialisation favorise l'optimisation de requêtes mais en contre partie elle entraîne un sur coût de maintenance des données en cas de mise à jour des données sources [THE98][BEL00]

De nombreux travaux traitent ces problématiques, nous pouvons distinguer deux axes principaux de recherche :

La maintenance incrémentale des vues matérialisées qui se propose de répercuter les mises à jour survenues au niveau des données sources sans recalculer complètement les vues ;

La sélection des vues à matérialiser qui propose des algorithmes permettant de déterminer une configuration de vues à matérialiser dans l'entrepôt de données de telle sorte que le coût d'exécution des requêtes soit optimal.

Après la sélection des vues matérialisées, toutes les requêtes définies sur l'entrepôt doivent être réécrites en fonction des vues disponibles. Ce processus est appelé réécriture des requêtes en fonction des vues [SRIV 96]. La réécriture des requêtes a attiré l'attention de nombreux chercheurs car elle est en relation avec plusieurs problèmes de gestion de données : l'optimisation de requêtes, l'intégration des données, la conception des entrepôts de données, etc. Le processus de réécriture des requêtes a été utilisé comme technique d'optimisation pour réduire le coût d'évaluation d'une requête.

Par exemple, à partir d'ORACLE 8i, le processus de réécriture de requête incorporé transforme une commande SQL de telle façon qu'elle puisse accéder aux vues matérialisées. Cet outil de réécriture permet de réduire significativement le temps de réponse pour des requêtes d'agrégation ou de jointure dans les grandes tables des entrepôts. Quand une requête cible une ou plusieurs tables de base pour calculer un agrégat (ou pour réaliser une jointure) et qu'une vue matérialisée contient les données requises, l'optimiseur d'oracle peut réécrire la requête d'une manière transparente pour exploiter la vue, et procurer ainsi un temps de réponse plus court.

La matérialisation des vues est une approche embarrassante du fait qu'elle nécessite une anticipation des requêtes à matérialiser. Or, les requêtes dans les environnements des entrepôts sont souvent ad hoc et ne peuvent pas toujours être anticipées.

4.2 Les index

Dans les systèmes de gestion de bases de données (SGBD), l'accès aux données est d'autant plus lent que la base de données est volumineuse. Un parcours séquentiel des données est une opération lente et pénalisante pour l'exécution des requêtes, notamment dans le cas des opérations de jointure où ce parcours doit souvent être effectué de façon répétitive. La création d'un index permet d'améliorer considérablement le temps d'accès aux données en créant des chemins d'accès directs.

Il existe deux types d'index :

☒ Les index primaires (clustered ou index groupants)

Les adresses contenues dans cet index sont triées suivant le placement physique sur disque des n-uplets composant la table indexée, peu de blocs disques sont parcourus et les requêtes de recherche sont ainsi résolues de manière efficace, souffre d'un coût de maintenance très élevé car il faut maintenir l'ordre du tri, dans une table, il peut y avoir au plus un index primaire.

☒ Les index secondaires (non-clustered ou index non-groupants)

Les adresses contenues dans un index primaire sont triées suivant le placement physique sur disque des n-uplets composant la table indexée, sont moins efficaces que les index primaires, mais moins coûteux au niveau de la maintenance, index secondaires sur une table donnée sont possibles.

Dans les entrepôts de données, nous devons faire la différence entre

☒ Les techniques d'indexation,

- ☒ La sélection des index.

4.2.1 Techniques d'indexation

Les principales techniques d'indexation utilisées dans les SGBD relationnels et les entrepôts de données :

- **Index en B-arbre**

Un B-arbre est une liste chaînée de noeuds dont la valeur est celle de l'index. Les feuilles de l'arbre font référence à :

- ☒ Une seule valeur, si cet index est construit sur un attribut clé des n-uplets de la table indexée
- ☒ Plusieurs valeurs, si cet index est construit sur un attribut non-clé des n-uplets de la table indexée

Cette référence spécifie l'emplacement physique du n-uplet sur le disque [BME72].

Un B-arbre offre un excellent compromis pour les opérations de recherche par clé et par intervalle, ainsi que pour les mises à jour. Ces qualités expliquent le fait que les B-arbres et leurs variantes soient systématiquement intégrés dans la plupart des SGBD.

La Figure 9 montre un exemple de B-arbre construit sur la table Personne définie par le schéma Personne (Pr_ID, Pr_Nom, Pr_Age,...).

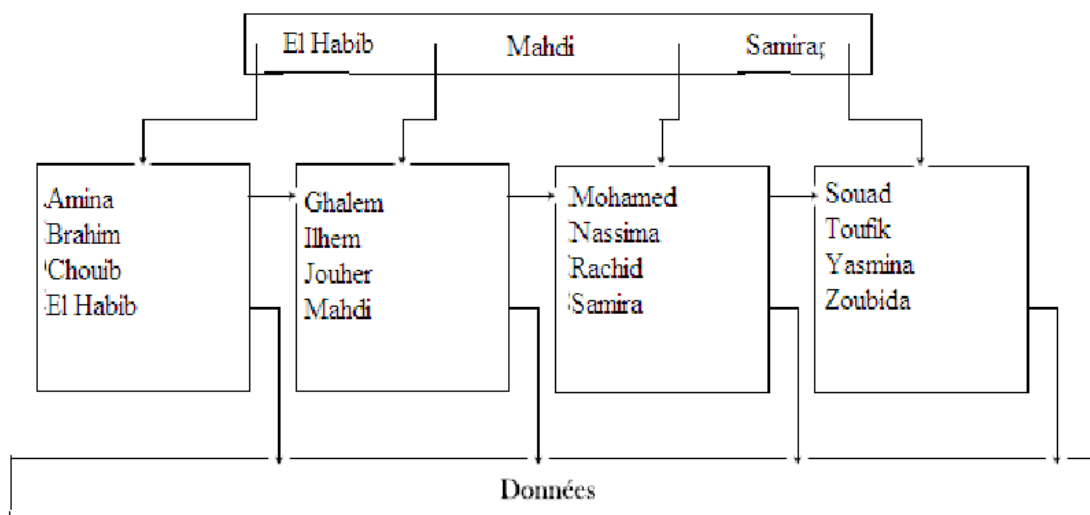


Figure 9 Index en B-arbre construit sur l'attribut Personne_Nom

- **Index de hachage**

Les tables de hachage sont des structures de données très couramment utilisées en mémoire centrale pour organiser des ensembles et fournir un accès performant à leurs éléments.

L'idée de base du hachage est d'organiser un ensemble d'éléments d'après une clé et d'utiliser une fonction, dite de hachage, qui, pour chaque valeur de clé c , donne l'adresse $f(c)$ d'un espace de stockage où l'élément doit être placé. En mémoire principale, cet espace de stockage est en général une liste chaînée et, en mémoire secondaire, un ou plusieurs blocs sur le disque.

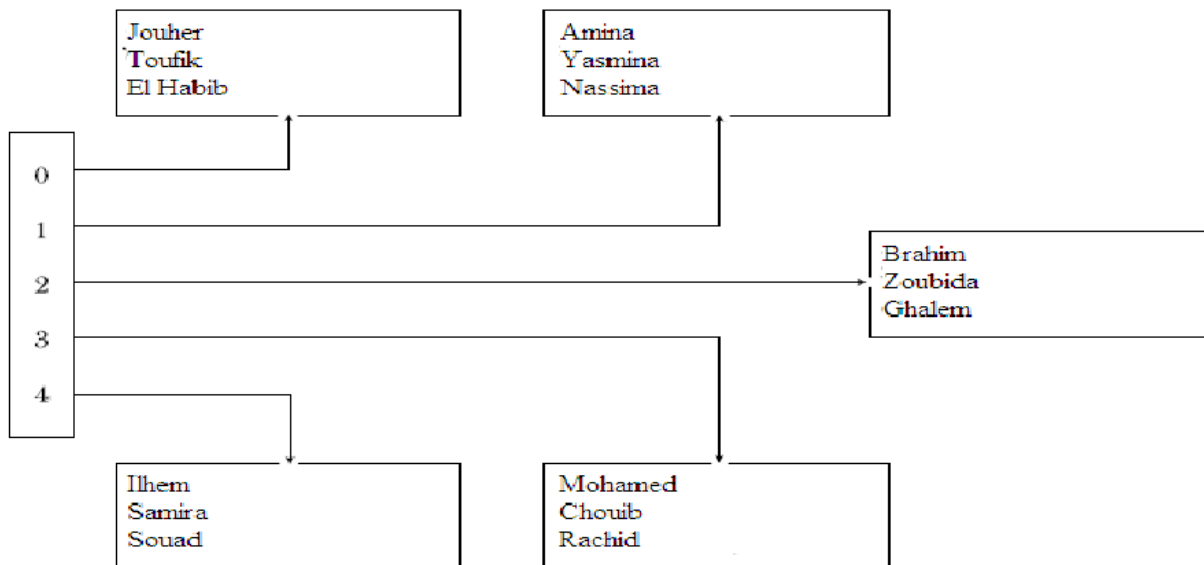


Figure 10 Index de hachage construit sur l'attribut Nom

La Figure 10 montre un exemple d'index de hachage construit sur la table Personne. La fonction de hachage est $H(\text{Nom}) = \text{rang}(\text{Nom}[0]) \bmod 5$, où $\text{Nom}[0]$ désigne la première lettre du Nom d'une Personne.

Une fonction de hachage mal conçue affecte tous les n -uplets à la même adresse et la structure dégénère vers un simple fichier séquentiel. Cela peut être le cas, avec notre fonction basée sur la première lettre du nom, pour tous les Personne dont le Nom commence par la lettre l.

- **Index bitmap**

Un index bitmap repose sur un principe très différent de celui des index en B-arbre.

Alors que dans ces derniers, on trouve, pour chaque attribut indexé, les mêmes valeurs dans l'index et dans la table, un index bitmap considère toutes les valeurs possibles de l'attribut indexé, que la valeur soit présente ou non dans la table [OQ97]. Pour chacune de ces valeurs possibles, un tableau de bits, dit bitmap, est stocké. Ce bitmap est composé d'autant de bits qu'il y a de n -uplets dans la table indexée. Notons par A l'attribut indexé et v la valeur définissant le bitmap. Chaque bit associé à un n -uplet a alors la signification suivante :

- si le bit est mis à 1, l'attribut A a pour valeur v pour ce n -uplet ;
- sinon, le bit est mis à 0.

Lorsque les n-uplets dont la valeur est v sont recherchés, il suffit donc de prendre le bitmap associé à v , de chercher tous les bits à 1 et d'accéder ensuite aux n-uplets correspondants. Un index bitmap est très efficace si le nombre de valeurs possibles de l'attribut indexé est relativement faible.

Un index bitmap est de très petite taille comparé à un B-arbre construit sur le même attribut. Il est donc très utile dans des applications de type entrepôt de données gérant de gros volumes de données et classant les informations par des attributs catégoriels définis sur de petits domaines de valeurs. Certaines requêtes peuvent alors être exécutées très efficacement, parfois sans même recourir à la table contenant les données.

Prenons l'exemple de la table Client et créons un index bitmap sur le sexe des personnes. La Figure 11 montre l'index bitmap pour les valeurs Féminin et masculin.

Table CLIENT				BM1	BM2
Nom	Age	...	Sexe	M	F
Mohamed	20		M	1	0
Amina	42		F	0	1
Omar	21		M	1	0
Othman	52		M	1	0
Aicha	18		F	0	1
Asmaa	17		F	0	1
rachid	36		M	1	0

Figure 11 Index bitmap construit sur le sexe des clients

- **Index de jointure**

L'opération de jointure est très coûteuse en terme de temps de calcul lorsque les tables concernées sont grandes. Plusieurs méthodes ont été proposées pour accélérer ces opérations. Ces méthodes incluent les boucles imbriquées, le hachage, la fusion, etc.

Valduriez [VALD 87] a proposé des index spécialisés appelés index de jointure, pour préjoindre des relations. Un index de jointure matérialise les liens entre deux relations par le biais d'une table à deux colonnes, contenant les RID (identifiant de n-uplet) des n-uplets joints deux par deux. Ce genre d'index est souhaité pour les requêtes des systèmes OLTP car elles possèdent souvent des jointures entre deux tables [REDB 97].

Par contre, pour les entrepôts de données modélisés par un schéma en étoile, ces index sont limités. En effet les requêtes décisionnelles définies sur un schéma en étoile possèdent plusieurs jointures. Il

faut alors subdiviser la requête en fonction des jointures. Or le nombre de jointures possibles est de l'ordre de $N!$, N étant le nombre de tables à joindre (problème d'ordonnancement de jointure).

Pour résoudre ce problème, Redbrick [REDB 97] a proposé un nouvel index appelé index de jointure en étoile, adapté aux requêtes définies sur un schéma en étoile. Un index de jointure en étoile peut contenir toute combinaison de clés étrangères de la table des faits.

Ce type d'index est dit complet s'il est construit en joignant toutes les tables de dimensions avec la table des faits. Un index de jointure partiel est construit en joignant certaines des tables de dimensions avec la table des faits. En conséquence, l'index complet est bénéfique pour n'importe quelle requête posée sur le schéma en étoile. Il exige cependant beaucoup d'espace pour son stockage.

4.2.2 Sélection d'index

A partir d'un ensemble de requêtes décisionnelles et la contrainte d'une ressource donnée (l'espace, le temps de maintenance, etc.) on sélectionne un ensemble d'index afin de minimiser le coût d'exécution des requêtes.

Le groupe base de données de Microsoft a développé un outil pour sélectionner des index avec Microsoft SQL Server 7.0 [CHAU 98]. L'architecture de l'outil de sélection des index proposé est illustrée ci-dessous.

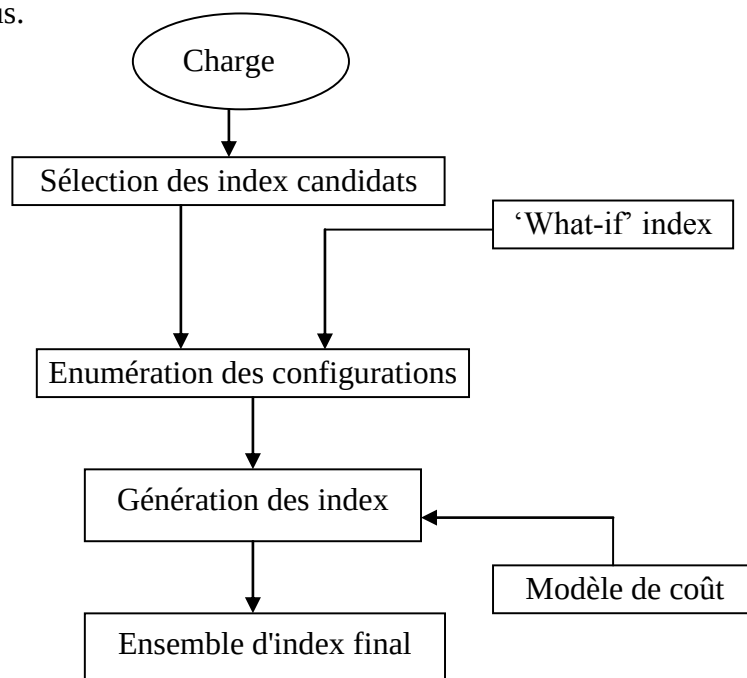


Figure 12 L'architecture de l'outil de sélection d'index

L'outil prend un ensemble de requêtes définies sur un schéma de base de données. Le traitement est itératif. Durant la première itération, il choisit les index sur une colonne (mono index) ; dans la deuxième les index sur deux colonnes et ainsi de suite. L'algorithme de recherche d'index est testé en fonction de ces trois modules:

- ☒ La sélection des index candidats,
- ☒ L'énumération des configurations,
- ☒ La génération des multi index.

Le module de sélection des index candidats permet de déterminer la meilleure configuration pour chaque requête d'une manière indépendante. Finalement, il fait l'union de ces configurations.

S'il existe n index candidats, et que l'outil doit sélectionner k parmi n index, le module d'énumération doit énumérer toutes les configurations, et à l'aide d'un modèle de coût sélectionner le meilleur ensemble de configurations garantissant un coût minimal.

Cet algorithme de sélection des index prend une requête à un moment donné et sélectionne tous les index possibles. Cependant, l'ensemble des index utilisant cette méthodologie pourra exiger beaucoup d'espace de stockage et des coûts de maintenance élevés.

Dans le but de minimiser les coûts de stockage et de maintenance, Chaudhuri et al. [CHAU 99] ont proposé une technique appelée fusion d'index (index merging). Elle prend un ensemble d'index ayant une capacité d'espace S et fournit un nouvel ensemble d'index ayant une capacité d'espace S_0 inférieure à celle de départ ($S_0 < S$). L'opération de fusion est guidée par un modèle de coût : la fusion est appliquée s'il y a une réduction dans le coût d'exécution des requêtes. La technique de fusion d'un ensemble d'index ressemble à la reconstruction des fragments verticaux d'une relation donnée.

- ☒ Tous les algorithmes proposés pour résoudre ces problèmes sont dirigés par un modèle de coût. Ce dernier permet non seulement de dire si une vue (ou index) est plus bénéfique qu'une autre vue (ou index), mais également d'orienter ces algorithmes dans leur sélection. En conséquence il faut prévoir un modèle de coût des requêtes pour mieux les optimiser. Le modèle de coût accepte en paramètre le plan d'exécution d'une requête et retourne son coût. Le coût d'un plan d'exécution est évalué en cumulant le coût des opérations élémentaires (sélection, jointure, etc.). Ces modèles de coûts contiennent, d'une part, des statistiques sur les données et, d'autre part, des formules pour évaluer le coût. Ces coûts sont mesurés en

unités de temps si l'objectif est de réduire le temps de réponse des requêtes, le nombre d'entrées-sorties ou le temps de maintenance des vues et des index.

- ☒ L'optimisation par index se fait d'une manière séquentielle ; c'est-à-dire, d'abord la sélection des vues matérialisées et ensuite la sélection des index. Cette façon de procéder ne prend pas en compte l'interaction entre les vues et les index et pose un problème de gestion de ressources. Par exemple, considérons que ces deux problèmes soient contraints par la capacité d'espace. Il s'agit alors de savoir comment distribuer l'espace entre les vues et les index afin de garantir une meilleure performance des requêtes ?

4.3 La fragmentation

La fragmentation est une technique, permettant l'optimisation de performances des requêtes et d'éviter le balayage de grandes tables, consiste à diviser un schéma de données en plusieurs fragments (sous schémas) de telle façon que la combinaison de ces fragments produit l'intégralité des données sources, sans perte ou ajout d'information. Le but est de réduire le temps d'exécution des requêtes [BBK05].

Les travaux qui traitent de la fragmentation dans les entrepôts de données relationnels s'inspirent de ceux proposés dans les bases de données relationnelles [NKR95][ZYO94]; et orientées objets [BKL98b][ECB95][RFZ95]. Cette fragmentation peut être horizontale, verticale ou mixte.

4.3.1 La fragmentation verticale

C'est une relation qui est divisée en sous relations appelées fragments verticaux qui sont des projections appliquées à la relation (Figure 13). La fragmentation verticale favorise naturellement le traitement des requêtes de projection portant sur les attributs utilisés dans le processus de la fragmentation, en limitant le nombre de fragments à accéder. Son inconvénient est qu'elle requiert des jointures supplémentaires lorsqu'une requête accède à plusieurs fragments.

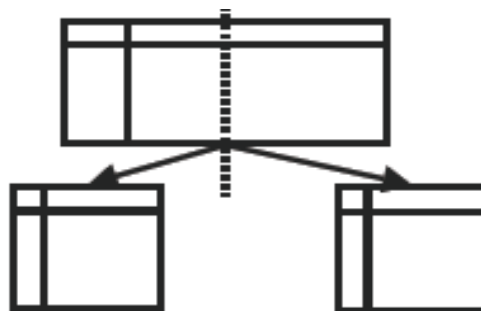


Figure 13 Fragmentation verticale

Les auteurs dans [DAT99] exploitent la fragmentation verticale pour construire un index nommé "Cuio" dans un entrepôt modélisé par un schéma en étoile. Cuio permet d'accélérer l'accès aux données et optimise l'espace de stockage en matérialisant les fragments au lieu des attributs indexés.

Afin de minimiser le temps de réponse des requêtes, Golfarelli et al. utilisent la fragmentation verticale pour partitionner des vues définies sur un entrepôt [GMR99]. Cette fragmentation est basée sur une charge de requêtes et un modèle de coût. Selon les auteurs, la fragmentation verticale désigne deux opérations : d'une part le partitionnement d'une vue en plusieurs fragments et, d'autre part, l'unification en une seule vue de deux ou plusieurs vues ayant une clé commune.

L'unification respecte la règle de reconstruction d'une table fragmentée à partir de ses fragments verticaux et vise à réduire la redondance des vues. Les auteurs supposent que leur approche peut être bénéfique pour la distribution de l'entrepôt sur une architecture parallèle et proposent de combiner leur algorithme de fragmentation avec un algorithme d'allocation des fragments sur les nœuds distants [OZS99].

Munneke et al. [MWM99], proposent un autre type de fragmentation, appelée *server*, qui est équivalente à une fragmentation verticale dans une base de données relationnelle. La fragmentation *server* élimine une ou plusieurs dimensions dans un cube pour produire un fragment. Afin d'assurer la reconstruction des fragments, une ou plusieurs dimensions sont dupliquées dans tous les fragments.

4.3.2 La fragmentation horizontale

Elle consiste à diviser une relation R en sous ensembles de n -uplets appelés fragments horizontaux, chacun étant défini par une opération de restriction appliquée à la relation (Figure 14). Les n -uplets de chaque fragment horizontal satisfont une clause de prédicats.

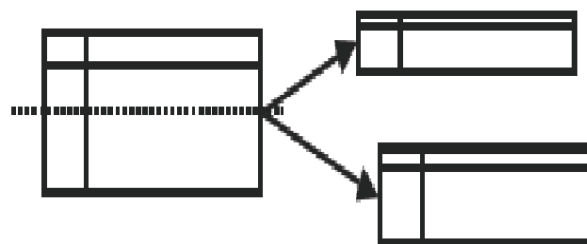


Figure 14 Fragmentation Horizontale

Tout algorithme de fragmentation horizontale nécessite la donnée d'un ensemble de requêtes les plus fréquentes. A partir de ces requêtes, on extrait deux types d'informations :

- ☒ Les informations qualitatives concernent les tables de dimension et sont représentées par les prédicats de sélection simples utilisés dans les requêtes.
- ☒ Les informations quantitatives concernent la sélectivité de ces prédicats et les fréquences d'accès des requêtes.

On rappelle qu'un prédicat simple p est défini par :

$$p : A_i \theta \text{ Valeur}$$

Où A_i est un attribut d'une relation à fragmenter, $\theta \in \{=, <, \leq, >, \geq, \neq\}$,
 $\text{Valeur} \in \text{Dom}(A_i)$.

Pour ce qui est de la fragmentation horizontale, certains auteurs ont proposé une technique de construction d'un entrepôt réparti en utilisant la stratégie descendante [OZSU 99]. Cette stratégie est utilisée pour la conception de bases de données réparties. Elle part du schéma conceptuel global d'un entrepôt, qu'elle répartit pour construire les schémas conceptuels locaux. Cette répartition se fait en deux étapes essentielles, à savoir, la fragmentation et l'allocation, suivies éventuellement d'une optimisation locale. Dans [BEK00][BES02], l'algorithme proposé de fragmentation horizontale d'un schéma en étoile se base sur un ensemble de requêtes de départ.

Noaman & Barker Afin de construire un entrepôt de données distribué, les auteurs exploitent une stratégie descendante par fragmentation horizontale [NBK99]. Elle part du schéma conceptuel global d'un entrepôt, qu'elle répartit pour construire les schémas conceptuels locaux. Cette répartition se fait en deux étapes essentielles : la fragmentation et l'allocation, suivies éventuellement d'une optimisation locale. Les auteurs proposent un algorithme qui dérive des fragments faits en se basant sur des requêtes définies sur les dimensions.

Bellatreche [BEL00] applique la fragmentation horizontale dérivée sur un schéma en étoile et propose plusieurs approches basées sur un ensemble de requêtes. L'auteur adapte les algorithmes proposés dans le contexte des bases de données réparties. Ces algorithmes se basent sur la complétude et la minimalité des prédicats ou sur les affinités des requêtes

[BEL00] constate que ces méthodes génèrent un nombre important de fragments et rendent ainsi leur processus de maintenance très coûteux. Pour répondre à cette problématique, il propose des algorithmes de sélection d'un schéma de fragmentation optimal. Ces algorithmes visent à trouver un

compromis entre le coût de maintenance des fragments et le coût d'exécution des requêtes. Ils sont basés sur des modèles de coût et procèdent en trois étapes :

- ☒ génération de plusieurs schémas de fragmentation,
- ☒ évaluation de ces schémas
- ☒ sélection d'un schéma optimal.

Le premier algorithme est exhaustif et consiste à construire tous les schémas de fragmentation possibles par fragmentation horizontale. Il énumère ensuite ces schémas et calcule pour chacun d'eux le coût d'exécution des requêtes de la charge.

Il sélectionne finalement le schéma qui correspond au coût minimum. Le deuxième algorithme est approximatif. Il construit un schéma initial par l'algorithme de fragmentation dirigée par les affinités, puis l'améliore par des opérations de fusion ou de décomposition des fragments. Finalement, le troisième algorithme [BBK05] exploite un algorithme génétique pour sélectionner un schéma de fragmentation que l'on va comparer avec notre contribution dans le chapitre conception.

4.3.3 La fragmentation mixte

La fragmentation mixte partitionne verticalement des fragments horizontaux ou horizontalement des fragments verticaux (Figure 15). Les algorithmes de fragmentation mixte ont été étudiés dans le contexte relationnel et sont subdivisés en deux types : la fragmentation par création de grille [NKR 95] et la fragmentation par définition de vues [PKN91].

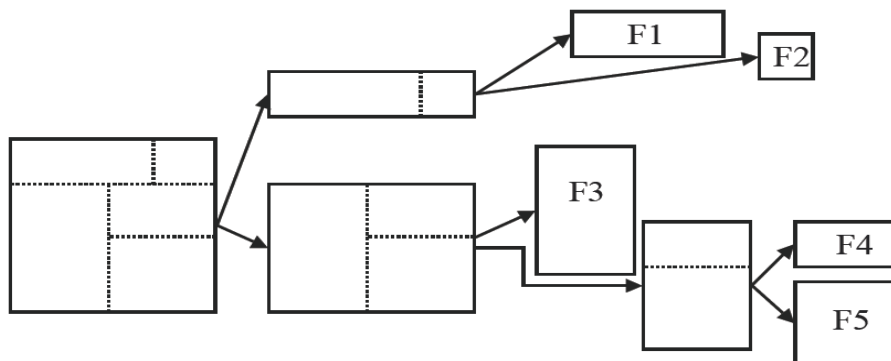


Figure 15 Fragmentation mixte

Wu et Buchmaan[WUA97]. Les auteurs recommandent de combiner la fragmentation horizontale et la fragmentation verticale. Selon eux, la table de faits peut être partitionnée horizontalement à partir de fragments définis sur les dimensions. Elle peut aussi être partitionnée verticalement selon les clés

étrangères faisant référence aux dimensions. Cependant, aucun algorithme de fragmentation n'est proposé.

4.3.4 Évolution de la fragmentation dans les SGBD commerciaux

Plusieurs SGBDs commerciaux ont proposé des langages de définitions de données pour supporter la fragmentation. À titre d'exemple, le SGBD Oracle offre plusieurs modes de partitionnements:

- ☒ Partitionnement par intervalle (Range) : définit par :
le tuple (c, V), où c est un type de colonne et V une séquence ordonnée de valeurs dans le domaine de c.
- ☒ Partitionnement par hachage (Hash) décompose la table selon une fonction de hachage (fournie par le système) appliquée sur les valeurs des attributs de fragmentation.
- ☒ Partitionnement par liste (List) décompose une table selon les listes de valeurs d'une colonne.
- ☒ Partitionnement composé (Range-Hash et Range-List) est utilisé à l'aide des instructions PARTITION-SUBPARTITION.
- ☒ Partitionnement par une colonne virtuelle (virtual column partitioning) dans lequel, une table est fragmentée en utilisant un attribut virtuel, qui est défini par une expression utilisant un ou plusieurs attributs. Cette colonne est stockée seulement dans les métadonnées.
- ☒ Le partitionnement par référence (referential partitioning) : qui permet de fragmenter une table en utilisant une autre table (à condition il y a une relation de type père-fils entre les deux tables). Ce partitionnement est similaire à la fragmentation dérivée, mais son inconvénient majeur est qu'une table est fragmentée en fonction d'une seule autre table alors que dans les entrepôts de données, une table des faits doit être fragmentée en utilisant les schémas de fragmentation de plusieurs tables de dimension pour garantir une optimisation des requêtes complexes.

5 Conclusion

Les entrepôts de données sont devenus maintenant non pas un phénomène de mode mais un instrument indispensable à la bonne marche de l'organisation. Ils sont en effet à la base de toute stratégie et prise de décision de l'entreprise. Il faut en extraire les informations nécessaires à la prise de décision et également leur structure. De cet entrepôt sont extraites des bases de données multidimensionnelles, car elles permettent de regarder l'organisation sous différents angles ou dimensions, ces dernières sont constituées que de données propres à la décision.

Dans ce chapitre, nous avons présenté les techniques principales utilisées pour optimiser le temps d'exécution des requêtes dans les entrepôts de données. Il s'agit de la fragmentation, des vues matérialisées, et de la création d'index. Ces deux dernières techniques sont redondantes dont la mise à jour est très coûteuses tandis que la fragmentation est non redondante et s'exécute en temps réel où les mises à jour n'auront pas lieu. Nous avons mis en évidence les problèmes liés à chaque technique. Vu que la fragmentation horizontale est une structure non redondante et ne duplique pas les données, elle est la plus utilisée, la plupart des algorithmes de fragmentation horizontale existants donne un nombre de fragments que l'on ne peut pas contrôler. Nous exposerons les méthodologies de la fragmentation horizontale dans les entrepôts de données dans le chapitre suivant.

Chapitre 2

*SCHEMA DE FRAGMENTATION HORIZONTAL
ET CALCUL DE COUT*

1 Introduction

Dans ce chapitre, nous présentons les différentes étapes pour aboutir à un schéma de fragmentation quasi optimal. Ce dernier doit permettre d'optimiser les performances de requêtes en se basant sur les techniques de fragmentation horizontale.

Elle constitue un aspect important dans la conception physique des bases de données. Elle a un impact significatif sur l'exécution des requêtes OLAP et la gestion de l'entrepôt de données. Dans le contexte des entrepôts de données relationnels, elle permet de partitionner les tables, les index et les vues matérialisées en plusieurs ensembles de lignes disjoints, physiquement stockés et séparément consultés [SNY04], [ZRL04], c'est une structure non redondante.

Contrairement aux vues matérialisées et aux index, la fragmentation horizontale ne duplique pas les données, par conséquent elle réduit le coût de mises à jour [PSA04]. Elle peut également être combinée avec d'autres structures d'optimisation comme les index, les vues matérialisées [BSL04] et le traitement parallèle [SMR00], [RZL02].

2 Méthodologie de fragmentation horizontale dans les entrepôts de données

Dans les entrepôts de données relationnels, deux types de tables : les tables de dimension et la ou les table(s) des faits sont candidates pour la fragmentation. Plusieurs scénarios de fragmentation sont possibles :

1) Fragmenter seulement les tables de dimensions en utilisant la fragmentation horizontale primaire. Ce choix n'est pas souhaitable pour les requêtes décisionnelles, pour deux raisons :

- ☒ les tailles des tables de dimensions sont généralement inférieures à la taille de la table des faits,
- ☒ les requêtes décisionnelles accèdent à la table des faits (qui est très volumineuse) par des opérations de jointures qui sont coûteuses [LRA98]. Par conséquent, toute fragmentation ne prenant pas en considération la table des faits est à exclure.

2) Partitionner seulement la table des faits en utilisant la fragmentation primaire. Notons que la table des faits est composée des clés étrangères des tables de dimensions et des mesures numériques, comme le montant des ventes, le nombre d'articles soldés, etc.. Généralement, dans une requête décisionnelle, nous trouvons rarement des prédicats de sélection définis sur des attributs d'une table des faits [NBK99]. De plus, les requêtes définies sur un schéma en étoile possèdent les caractéristiques suivantes [OPQ97]:

- ☒ des opérations de jointures multiples entre la table des faits et les tables de dimension,
- ☒ pas de jointure entre les tables de dimensions
- ☒ chaque table de dimension impliquée dans une opération de jointure a plusieurs prédicats de sélection sur ses attributs.

3) Fragmenter totalement ou partiellement les tables de dimensions en utilisant la fragmentation primaire et utiliser leurs schémas de fragmentation pour partitionner la table des faits. Ce choix est bien adapté aux entrepôts, car il prend en considération les exigences des requêtes décisionnelles, ainsi que les relations entre les tables de dimensions et la table des faits [BEL00]. Concrètement, elle consiste d'abord à partitionner certaines/toutes les tables de dimension en utilisant leurs prédicats de sélection simples, ensuite à fragmenter la table de faits en utilisant les schémas de fragmentation des tables de dimension. Cette procédure de fragmentation prend en compte la caractéristique des requêtes en étoile entre la table des faits et les tables de dimension. Afin d'illustrer la procédure de fragmentation horizontale dans les entrepôts de données relationnels, nous exposons l'exemple tiré de [BEL00].

Étant donné un schéma en étoile avec trois tables de dimension : Client, Temps et Produit et une table de faits Ventes.

La table de dimension Client est fragmentée en deux fragments Client₁ et Client₂ définis par les clauses suivantes :

Client₁ = $\sigma_{\text{Sexe} = 'M'}(\text{Client})$

Client₂ = $\sigma_{\text{Sexe} = 'F'}(\text{Client})$

Ce type de fragmentation est supporté par les SGBDs commerciaux. Cette fragmentation est matérialisée par la clause SQL suivante (variante de SQL Oracle [BAE05]) :

```
CREATE TABLE Client (NC number(4), Cnom varchar2(14), Sexe varchar2(1)) PARTITION BY
LIST (Sexe)
(PARTITION Client1 values ('M'), PARTITION Client2 values ('F')) ;
```

La table de faits Ventes peut être fragmentée en deux fragments Ventes₁ et Ventes₂ définis par :

Ventes₁ = Ventes $\bowtie$ Client₁

Ventes₂ = Ventes $\bowtie$ Client₂,

Où $\bowtie$ représente l'opération de semi jointure

Ce processus de fragmentation génère deux sous schémas en étoile S_1 et S_2 tels que :

S_1 : (Ventes₁, Client₁, produit, temps) (activités de ventes réalisées par les clients masculins seulement).

S_2 : (Ventes₂, Client₂, produit, temps) (activités de ventes réalisées par les clientes seulement).

La table des faits est alors fragmentée en utilisant la fragmentation dérivée.

Pour le calcul de la complexité de la procédure de fragmentation, on prend un schéma en étoile de d tables de dimension et une table des faits. Soit g ($g \leq d$) le nombre de tables de dimensions horizontalement fragmentées. Le nombre de fragments horizontaux (dénotté par N) de la table des faits est donné par l'équation suivante :

$$N = \prod_{i=1}^g m_i \quad \text{Où } m_i \text{ représente le nombre de fragments de la table de dimensions } D_i.$$

D'après l'équation ci-dessus, on constate que la fragmentation dérivée de la table des faits peut générer un très grand nombre de fragments et en conséquence beaucoup de sous schémas en étoile [BEL00].

Par Exemple considérons le schéma en étoile de l'exemple précédent, telles que les tables de dimensions soient partitionnées comme suit :

- CLIENT en 48 fragments en utilisant l'attribut "Wilaya" (48 wilaya en Algérie) ,
- TEMPS en 36 fragments en utilisant l'attribut "Mois"
- PRODUIT en 80 fragments en utilisant l'attribut type de produit.

La table des faits est donc fragmentée en 138 240 ($= 48 \times 36 \times 80$) fragments et le schéma en étoile en 138 240 sous schémas. Il devient difficile pour l'administrateur de l'entrepôt de gérer tous ces fragments.

A travers cet exemple, il apparaît indispensable de réduire le nombre de fragments de la table des faits pour une meilleure gestion. Il est donc nécessaire d'introduire des méthodes de sélection des tables de dimension à fragmenter pour [BEL00]:

- ☒ Éviter l'explosion du nombre de fragments de la table des faits, pour cela l'administrateur de l'entrepôt a la possibilité de choisir le nombre maximum de fragments (W). Ce nombre est fixé en fonction du coût de maintenance de l'entrepôt fragmenté.
- ☒ Garantir une meilleure performance d'exécution d'un ensemble de requêtes identifiées comme les plus fréquentes, donc c'est la possibilité de faire augmenter le nombre de fragments tant que la performance globale peut être améliorée

Le problème est donc de trouver un compromis entre le coût de maintenance et le coût d'exécution des requêtes. Alors le problème de sélection d'un schéma de fragmentation horizontale se pose.

Des méthodologies de fragmentation dans le contexte des entrepôts de données (comme Noaman et al. [NBK99], ont été proposées pour tous les travaux sur la fragmentation horizontale dans les bases de données traditionnelles ou les entrepôts de données en adaptant les travaux d'Özsu [OMT99], des algorithmes de fragmentation d'un schéma relationnel ([BKA00], [OMT99], [NBK99]).

L'objectif principal de ces algorithmes est la réduction du coût d'exécution de requêtes fréquentes définies sur l'entrepôt de données ou sur la base de données répartie.

Dans un SGBD supportant la fragmentation horizontale, chaque requête globale (définie sur les tables de bases) ayant une conjonction de prédicats de sélection Q doit être réécrite en fonction des fragments horizontaux, où chacun est défini par une conjonction de prédicats P (ce processus est appelé, la localisation des fragments dans le contexte des bases de données réparties [OMT99]). Dans ce cas, l'optimiseur de requête doit identifier le(s) fragment(s) pertinent(s) pour répondre à cette requête. Trois scénarios sont possibles pour réécrire la requête sur les fragments [GWW96]:

- ☒ Pas de réécriture : dans ce scénario la conjonction de P et Q ($P \wedge Q$) est non satisfiable. Dans ce cas, le fragment représenté par P ne contribue pas à la réponse de la requête (fragment non pertinent),
- ☒ Réécriture parfaite : Dans ce scénario $P \rightarrow Q$. Par conséquent, tout le fragment participe dans la réponse de la requête.
- ☒ Réécriture partielle : Dans ce scénario P ne contredit pas Q et n'implique pas Q . Par conséquent, certains tuples de fragment contribuent à la réponse de la requête.

Le nombre de fragments horizontaux générés par tout algorithme de fragmentation a une incidence sur le processus de réécriture de requêtes en fonction des fragments, vu la présence des opérations de sélection dans les requêtes de jointure en étoile. Alors, le problème de la fragmentation horizontale est formulé de la façon suivante : étant donné :

- ☒ un ensemble de tables de dimension $D = \{D_1, D_2, \dots, D_d\}$ et une table de faits
- ☒ un ensemble de requêtes OLAP fréquentes $Q = \{Q_1, Q_2, \dots, Q_m\}$, où chaque requête Q_i

$(1 \leq i \leq m)$ possède une fréquence d'accès

- ☒ un seuil (W fixé par l'administrateur de l'entrepôt de données (AED) représentant le nombre maximum de fragments qu'il peut maintenir. La satisfaction de cette contrainte évite une explosion du nombre de fragments de la table de faits.

Le problème de fragmentation horizontale consiste à :

- ☒ déterminer un ensemble de tables de dimension à fragmenter
- ☒ utiliser leurs schémas de fragmentation pour fragmenter la table de faits F en un ensemble de N fragments horizontaux (appelés les fragments de faits) tels que le coût total d'exécution des requêtes sur le schéma en étoile fragmenté soit réduit et la contrainte de maintenance soit satisfaite ($N \leq W$). Notons que le nombre de schémas de fragmentation possibles d'un entrepôt de données peut être très grand.

2.1 Processus de génération de schéma

Nous devons réaliser les tâches suivantes :

- ☒ L'extraction de tous les prédicats de sélection simples utilisés par les n requêtes.
- ☒ L'attribution à chaque table de dimension D_i ($1 \leq i \leq d$) d'un ensemble de prédicats simples EPS_{D_i} qui lui est propre.
- ☒ Toute table de dimension D_i ayant $EPS_{D_i} = \emptyset$, ne sera pas fragmentée. Soit $D_{candidat}$ l'ensemble de toutes les tables de dimension ayant leur ensemble de prédicats simples non vide. Soit g la cardinalité de l'ensemble $D_{candidat}$ ($g \leq d$).
- ☒ L'application de l'algorithme COM_MIN (annexe 1) [OMT99] à chaque ensemble de prédicats simples d'une table de dimension D_i fournit en sortie une forme complète et minimale de ces ensembles. La règle de complétude et de minimalité assure que si une table est fragmentée en au moins deux fragments, elle sera accédée différemment par au moins une application [OMT99].

2.2 Représentation des fragments horizontaux

L'analyse des clauses représentant les fragments horizontaux permet d'effectuer une partition du domaine des attributs de fragmentation en sous domaines appelés sous domaines stables (SDS) [SAM94]. Les éléments de cette partition sont déterminés par les prédicats simples.

On note que chaque fragment est représenté par une clause conjonctive de prédicats simples. Finalement, un schéma de fragmentation doit présenter les trois propriétés suivantes [BEL00].

- ☒ La complétude assure que chaque instance de l'ensemble fragmenté appartient à au moins un fragment.
- ☒ La reconstruction garantit la reconstitution de l'ensemble de données sources sans perte ni ajout d'information.
- ☒ La disjonction indique que les fragments doivent être disjoints deux à deux.

2.3 Identification des fragments participants à une requête

Dans un schéma en étoile fragmenté, on doit assurer l'accès transparent à l'utilisateur de l'entrepôt. Dans ce but, la tâche de l'optimiseur des requêtes (interrogation ou mise à jour) est de transformer les requêtes globales (c'est-à-dire les requêtes définies sur le schéma en étoile non fragmenté) en requêtes locales (définies sur le schéma en étoile fragmenté). Avant d'exécuter une requête sur un schéma fragmenté, l'optimiseur doit identifier le(s) sous schéma(s) en étoile satisfaisant cette requête. Ces derniers sont appelés les schémas valides. Une fois ces sous schémas sélectionnés, l'unité d'exécution de cette requête devient un sous schéma au lieu du schéma global.

Exemple : Supposons que nous ayons un schéma en étoile fragmenté en 2 sous schémas en étoile {VENTES × Client_1, VENTES × Client_2} (Table II.1), soient les informations concernant les conditions de fragmentation de chaque table de dimensions et de la table des faits qui peuvent être représentées par une table qu'on appelle table de spécification de fragmentation du schéma. Cette table a trois colonnes : la première contient la liste des noms des tables de dimensions fragmentées et la table des faits, la deuxième fournit les fragments de chaque table de dimensions et la troisième spécifie les conditions de la fragmentation de chaque table (table de dimensions ou table des faits).

Dimension	Fragments	Condition de Fragmentation
CLIENT	Client_1 Client_2	Sexe = "M" Sexe = "F"
VENTES	Ventes_1 Ventes_2	VENTES × Client_1 VENTES × Client_2

Tableau 1 La table de spécification de la fragmentation

Si une requête contenant un attribut de fragmentation dans la clause WHERE, l'optimiseur dirige automatiquement la requête vers les partitions valides : nous fragmentons une table CLIENT en utilisant l'attribut SEXE et si cette dernière possède une condition sur cet attribut, l'optimiseur ne charge que la partition pertinente (VENTES × Client_1).

3 Modèle de coût

Pour valider les observations décrites précédemment, on développe un modèle de coût qui peut être considéré comme une mesure de performance de la fragmentation dans les entrepôts.

3.1 Composantes d'un modèle de coût

Le coût total pour évaluer une requête dans un SGBD se décompose en trois coûts principaux:

- ☒ le coût des entrées-sorties (IO) pour lire et écrire entre la mémoire et le disque,
- ☒ le coût CPU
- ☒ le coût COM de communication sur le réseau (si les données sont réparties sur le réseau).

Ce dernier est généralement exprimé en fonction de la qualité totale des données transmises.[APE88],[BKL98a] Il dépend de la topologie du réseau.

Les modèles de coût utilisés par les optimiseurs ont généralement les mêmes composantes [GRU96], [NAA99] :

- ☒ des statistiques des données stockées dans une méta base,
- ☒ des formules pour évaluer la cardinalité des résultats intermédiaires et des requêtes.

Le modèle du coût a comme entrée un plan d'exécution d'une requête et comme sortie le coût de ce plan.

Le coût d'un plan d'exécution est évalué en cumulant le coût des opérations élémentaires (la sélection, la projection, la jointure, etc.). Une fonction de coût est associée à chaque opérateur élémentaire. Pour établir avec précision une fonction de coût on doit connaître à l'avance les nombreux paramètres dont dépend l'opérateur ainsi que le détail de l'algorithme qui l'implémente.

Par exemple, en voulant évaluer le coût d'une requête ayant une opération de jointure entre deux tables R et S. les paramètres dont l'opérateur de jointure dépend sont : la taille des deux tables, le nombre de pages occupées par ces deux tables, la sélectivité des prédicats de jointure, les méthodes d'accès déterminant la nature de l'algorithme de jointure utilisée [RAM98](jointure avec boucles imbriquées, jointure en utilisant un index, jointure par fusion, etc.). La prise en compte de ces paramètres rend l'estimation de l'opération de jointure entre R et S plus facile.

3.2 Statistiques et estimations

L'optimiseur des requêtes se base sur les statistiques des données stockées pour générer les plans d'exécution des requêtes. La disponibilité de statistiques pertinentes peut largement améliorer la qualité des plans choisis par l'optimiseur [CVN00]. Par contre, l'absence de statistiques peut donner de mauvais plans d'exécution. Le groupe DMX (Data Management Exploration) de Microsoft Research a proposé des techniques d'automatisation des statistiques [CVN00]. Ces dernières sont disponibles dans le catalogue du système [CHA82],[YCC84]. Ces statistiques peuvent être divisées en trois catégories :

- ☒ Les statistiques décrivant les tables et les distributions des valeurs d'attributs : le nombre n-uplets et leur taille moyenne pour chaque table, le nombre de valeurs distinctes pour chacun des attributs (qui permet de calculer la sélectivité des opérations de sélection contenant des prédicats d'égalité), les valeurs minimales et maximales des attributs de sélection (pour calculer la sélectivité des prédicats de comparaison). La représentation la plus simple est celle qui suppose l'uniformité de distribution des valeurs et l'indépendance des attributs. Ces paramètres sont pris en compte par les systèmes commerciaux.
- ☒ Les statistiques décrivant les paramètres de la machine utilisée : taille des pages disque et mémoire, taille mémoire, coût moyen d'une entrée-sortie, etc.
- ☒ Les statistiques des méthodes d'accès : les index sont des structures persistantes, il faut donc estimer leurs tailles, le nombre de pages occupées par ces index, etc.

La sélectivité des prédicats (de sélection et de jointure) est un paramètre primordial pour estimer le coût des requêtes. Elle est définie comme un coefficient représentant le nombre d'objets sélectionnés rapporté à un nombre d'objets total d'une table. Si la sélectivité vaut 1, tous les objets sont sélectionnés. Si elle vaut 0, aucun objet n'est sélectionné.

Une méthode basée sur l'échantillonnage (sampling) [HNS 95] permet d'estimer la sélectivité des prédicats en exécutant la requête sur une portion des tables. Suivant les résultats obtenus sur ces petites portions, la sélectivité de la requête peut être estimée. Le problème dans un tel système est qu'il faut optimiser partiellement la requête pour pouvoir exécuter quelques parties. Ce système ne fonctionne que pour des requêtes simples et très coûteuses. En effet, le processus de l'échantillonnage doit être très efficace par rapport à l'exécution globale de la requête pour obtenir un gain significatif en performance [BEL00].

4 Conclusion

Pour assurer une bonne conception physique des entrepôts de données, il faut utiliser des structures redondantes comme les vues matérialisées et l'indexation et des structures non redondantes comme la fragmentation des données. Dans ce chapitre, nous nous sommes intéressés à la fragmentation horizontale des entrepôts de données relationnels, nous avons présenté une méthodologie complète pour la fragmentation horizontale dans les entrepôts de données modélisés par des schémas en étoile, on appelle ce formalisme « Problème d'optimisation ».

Dans le prochain chapitre nous allons adopter une approche hybride combinant l'algorithme tabou avec séparation/évaluation par le calcul de coût.

Chapitre 3

*CONCEPTION DU SCHEMA DE
FRAGMENTATION QUASI OPTIMAL*

1 Introduction

Nous présentons la démarche retenue pour la fragmentation horizontale afin d'améliorer les performances de requêtes. L'algorithme proposé se base sur l'algorithme Tabou combiné avec séparation / évaluation, notre étude est inspirée des travaux présentés dans les articles [BBK05], [BBA06], les résultats obtenus feront l'objet de comparaison avec ceux de l'algorithme génétique.

2 Algorithme Tabou

L'algorithme Tabou est adopté pour un affinage du choix de la variable à traiter par une recherche locale, pour permettre de sortir des minima locaux de manière efficace (intensification dans une zone de l'espace de recherche) et pour d'autres raisons tels que:

- ☒ Stratégie d'intensification : on mémorise les meilleures solutions rencontrées et l'on essaie d'en dégager quelques propriétés communes pour définir des régions intéressantes vers lesquelles on oriente la recherche.
- ☒ Stratégie de diversification: c'est le contraire de l'intensification. L'application de cette politique conduit à mémoriser les solutions les plus fréquemment visitées et imposer un système de pénalités, afin de favoriser les mouvements les moins souvent utilisés.
- ☒ Aspiration: il arrive qu'un mouvement en principe interdit, se révèle intéressant. Le critère d'aspiration le plus fréquemment utilisé et le plus simple, consiste à regarder si le mouvement considéré ne conduit pas à une solution de coût inférieur à celui de la meilleure solution rencontrée jusqu'à présent.
- ☒ Taille de la liste Tabou: la taille de la liste tabou est à déterminer empiriquement, elle varie avec les problème, mais c'est une donnée primordiale. En effet une liste trop petite peut conduire à un cycle, alors qu'une liste trop grande peut interdire des transformations intéressantes (on trouve des règles statiques et des règles dynamiques).

- ☒ Sélection du meilleur voisin: la politique de Best Fit, et la politique du First Fit (on sélectionne le premier voisin qui satisfait les contraintes tabous). La dernière politique est souvent utilisée lorsque la taille du voisinage ne permet pas d'effectuer une évaluation complète.
- ☒ Critères d'arrêt: sert à déterminer le moment où l'on considère que la solution trouvée est d'assez bonne qualité pour être recevable.
- ☒ La méthode Tabou, plus jeune que le recuit simulé donne quant elle est bien appliquée de meilleurs résultats. Elle est moins sensible aux paramètres de réglage.

La méthode Tabou a été présentée pour la première fois par F. Glover (« future Pathsfor Integer Programming and Linksto Artificial Intelligence », 1986). L'idée de base consiste à introduire la notion d'histoire dans la politique d'exploration des solutions. Le nom de baptême de recherche tabou donné en 1986 par Glover exprime l'interdiction de reprendre des solutions récemment visitées.

La recherche tabou (notée TS pour Tabu Search) [[GAL97] est une méthode de recherche locale très puissante ayant obtenu de bons résultats pour de nombreux problèmes combinatoires. ([JAL96]; [MAL97]).

Un mouvement permet de passer d'une solution valide à une autre. On dit que la nouvelle solution est un voisin de la précédente. L'ensemble des solutions que l'on peut atteindre à partir d'une solution s'appelle le voisinage.

A chaque itération, tous les mouvements possibles sont examinés et le "meilleur", ou plus exactement le moins mauvais est sélectionné. Comme cette manière d'agir peut cycler, c'est-à-dire répéter indéfiniment la même suite de mouvements, les derniers mouvements effectués sont considérés comme interdits ("tabou"), leur nombre représentant la taille de la liste tabou. Le mouvement effectivement choisi à chaque itération, est donc le meilleur mouvement non tabou.

Le principe tabou est relativement simple. Dans une première phase, la méthode part d'une configuration quelconque A appartenant à l'ensemble S (l'espace de recherche) et se déplace vers une configuration A' située dans le voisinage de A. L'algorithme explore

donc itérativement l'espace des configurations S . Afin de choisir le meilleur voisin, l'algorithme évalue la fonction Objectif en chaque point du voisinage, et conserve le voisin ayant la valeur la plus optimale.

L'originalité de TS réside dans le fait que la valeur de la fonction objectif du voisin retenu peut être plus mauvaise que celle de la configuration courante. Ce critère, autorisant des dégradations de la fonction objectif, évite que TS ne soit piégée dans un optimum local mais induit un risque de cyclage. En effet, lorsque TS a quitté un optimum quelconque suite à une dégradation de la fonction objectif, elle peut revenir sur ses pas, à l'itération suivante. Pour pallier ce problème, une mémoire lui est adjointe afin de conserver pendant un certain temps la trace des dernières configurations déjà visitées. Ces configurations sont déclarées tabou et sont stockées dans une liste de longueur λ donnée, appelée liste tabou. Une nouvelle configuration n'est acceptée que si elle n'appartient pas à cette liste taboue. Ce critère d'acceptation d'une nouvelle configuration évite le cyclage de la méthode, durant la visite d'un nombre de configurations au moins égal à la longueur de la liste tabou, et il dirige l'exploration vers des régions de l'espace de recherche encore non visités [LAR05].

La liste Tabou est généralement gérée comme une liste de type FIFO : la configuration Tabou la plus ancienne est éliminée à chaque itération et elle est remplacée par la nouvelle configuration retenue. Mais le codage d'une telle liste est encombrant, car il faudrait garder en mémoire toutes les configurations. Pour pallier cette contrainte, la liste tabou des configurations interdites est remplacée par une liste d'opérations "interdites" représentant les opérations ayant permis de passer d'une configuration à une autre. Le remplacement de la liste tabou des configurations visitées par la liste des opérations conduit non seulement à l'interdiction de revenir vers des configurations précédentes (évite le cyclage court), mais aussi vers un ensemble de configurations dont plusieurs peuvent ne pas avoir été visitées jusqu'ici. Il est donc indispensable de corriger ce défaut et de trouver un moyen de lever l'interdiction d'accepter une opération déjà effectuée (donc appartenant à la liste tabou), par le biais d'un critère, appelé critère d'aspiration. Le critère d'aspiration le plus simple et le plus couramment utilisé consiste à tester si la configuration produite par une opération de statut tabou présente un coût inférieur à celui de la meilleure configuration trouvée jusqu'à présent. Si cette situation se produit, le statut tabou de l'opération est levé. Ce critère est évidemment très strict et ne devrait pas être satisfait très souvent, par conséquent, il apporte peu de changements dans le fonctionnement de la méthode. D'autres critères

d'aspiration plus complexes peuvent être envisagés. L'inconvénient de recourir trop souvent à l'aspiration est qu'elle peut altérer, d'une certaine façon, la protection offerte par la liste Tabou vis-à-vis du risque de bouclage.

TS utilise une stratégie de recherche agressive pour exploiter son voisinage. Il est donc crucial de mettre en place des structures de données et des techniques spécifiques permettant une mise à jour rapide des évaluations des mouvements et réduisant l'effort nécessaire pour trouver le meilleur mouvement d'où l'approche Séparation / Évaluation.

3 Algorithme Séparation / Évaluation

Un algorithme par séparation / évaluation, également appelé selon le terme anglo-saxon « Branch and Bound », est une méthode générique de résolution de problèmes d'optimisation, et plus particulièrement de complexité difficile. C'est une méthode d'énumération implicite (toutes les solutions possibles du problème peuvent être énumérés mais, l'analyse des propriétés du problème permet d'éviter l'énumération de classe de mauvaises solutions).

Le principe de l'algorithme est de décomposer un problème donné en deux ou plusieurs sous problèmes de tailles inférieures.

Suivant une stratégie définie préalablement, un sous problème est sélectionné puis partagé, sauf si on peut prouver que les sous problèmes résultants ne peuvent conduire à une solution optimale, ou qu'ils ne peuvent être décomposés.

Afin de choisir un problème parmi ceux qui n'ont pas été examinés, une évaluation (borne inférieure) des solutions dans chaque sous problème est introduite. Un sous problème dont l'évaluation est supérieure à la valeur de la meilleure solution connue peut être supprimé.

4 Mise en œuvre de la démarche

Afin de mettre en œuvre notre proposition d'algorithme tabou combiné avec séparation/évaluation, nous étions obligé de ré implémenter les modules tels que décrit dans l'organigramme ci-dessus avec intégration de notre solution qui se situe au niveau du module

" Algorithme évaluation des requêtes pour chaque schéma". Nous présentons dans ce qui suit la description des modules.

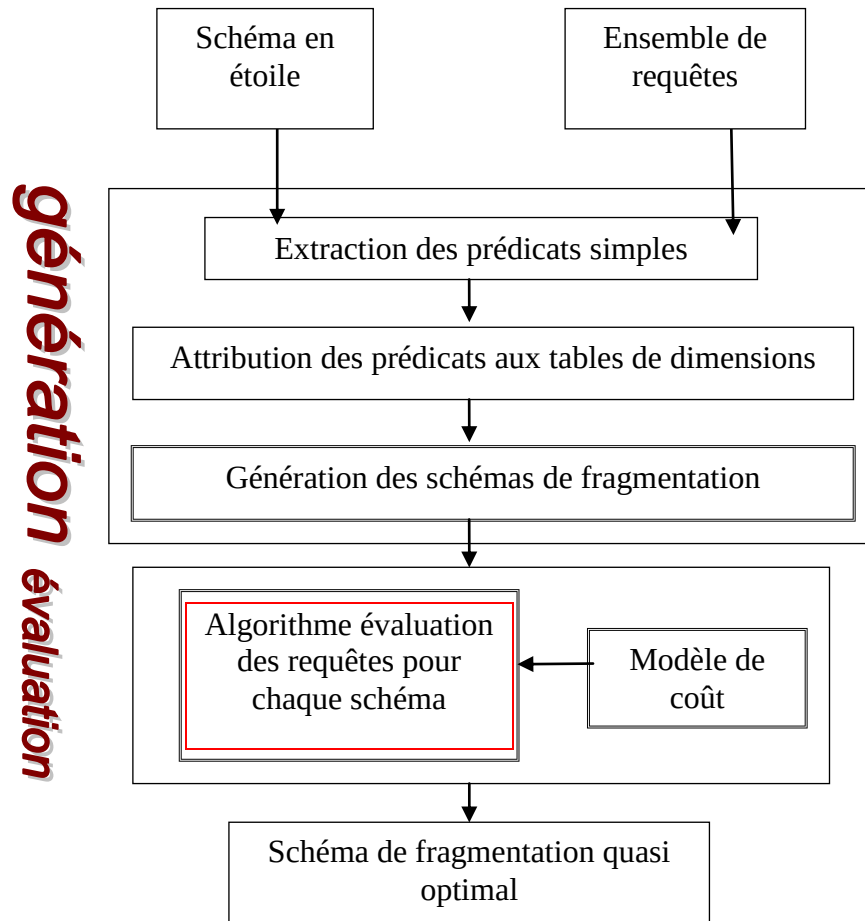


Figure 16 Organigramme de l'application

4.1 Le générateur de schémas

Les fragments horizontaux sont obtenus à partir des prédicats de sélection des requêtes définies sur la table à fragmenter. Le générateur de schémas de fragmentation consiste à générer tous les schémas de fragmentation possibles de la table T en utilisant les minterms obtenus à partir des prédicats de sélection définis sur T .

Un schéma de fragmentation représente le résultat du processus de fragmentation. Un schéma de fragmentation S d'une table T ayant m fragments horizontaux $\{F_1, F_2, \dots, F_m\}$ est dénoté

par : $S : ((F_1 : cl_1), (F_2 : cl_2), \dots, (F_m : cl_m))$, où chaque fragment F_i ($1 \leq i \leq m$) est défini par une clause de prédicats cl_i .

Le processus de génération de tous les schémas de fragmentation possibles est réalisé en trois étapes principales :

- ☒ l'énumération de l'ensemble des prédicats simples : nous énumérons l'ensemble de tous les prédicats de sélection définis sur la table T , à savoir $\{ \text{PR}_1, \text{PR}_2, \dots, \text{PR}_n \}$. Ces prédicats doivent être minimaux et mutuellement exclusifs [OV91].

- ☒ la génération de l'ensemble des minterms : à partir des prédicats de l'ensemble Π , l'ensemble de tous les minterms M est généré de la manière suivante :

$$M = \{ \text{PR}_1, \text{PR}_2, \dots, \text{PR}_n \}$$

- ☒ la combinaison de minterms : si nous faisons l'analogie entre les minterms et les fragments d'un schéma de fragmentation, nous pouvons dire qu'un minterms est une clause d'un fragment horizontal, d'où la propriété suivante.

Propriété : Un minterm est la conjonction de prédicats de sélection de P et en utilisant les opérateurs $\wedge$ et $\neg$ [NYS97]. Soit l'ensemble de prédicats $P = \{p, p'\}$, un minterm m peut être exprimé par l'expression $p \wedge p'$ ou $p \wedge \neg p'$.

En conséquence, le nombre de tous les schémas de fragmentation possibles d'une table donnée est le nombre de toutes les combinaisons possibles de minterms.

Exemple : Pour illustrer les trois étapes du processus de génération de schémas de fragmentation, considérons l'exemple suivant :

Supposons que nous ayons six prédicats simples et disjoints $\{p_1, p_2, \dots, p_6\}$ définis sur la table de dimension CLIENT. Les minterms de prédicats générés.

Prédicats	Description
P1	Age $\leq$ 40
P2	Age $\geq$ 40
P3	Sexe='M'
P4	Sexe='F'
P5	Ville='Oran'
P6	Ville='Alger'

Tableau 2 les six prédicats

À partir de ces prédicats les minterms sont :

- m1 (Age $\leq$ 40) $\wedge$ (Sexe='M') $\wedge$ (Ville= 'Oran'),
- m2 (Age $\leq$ 40) $\wedge$ (Sexe='M') $\wedge$ (Ville= 'Alger'),
- m3 (Age $\leq$ 40) $\wedge$ (Sexe='F') $\wedge$ (Ville= 'Oran'),
- m4 (Age $\leq$ 40) $\wedge$ (Sexe='F') $\wedge$ (Ville= 'Alger'),
- m5 (Age $>$ 40) $\wedge$ (Sexe='M') $\wedge$ (Ville= 'Oran'),
- m6 (Age $>$ 40) $\wedge$ (Sexe='M') $\wedge$ (Ville= 'Alger'),
- m7 (Age $>$ 40) $\wedge$ (Sexe='F') $\wedge$ (Ville= 'Oran'),
- m8 (Age $>$ 40) $\wedge$ (Sexe='F') $\wedge$ (Ville= 'Alger'),

Parmi les 26 minterms possibles, nous avons éliminé ceux qui sont contradictoires, par exemple : $p1 \wedge p2 \wedge p3 \wedge p4 \wedge p5 \wedge p1$. Les minterms résultats sont appelés les minterms primitifs.

Chaque minterm primitif $m_i (1 \leq i \leq 8)$ définit un fragment possible. Si c'est le cas, nous aurons un schéma de fragmentation de huit fragments (ce qui représente le même schéma obtenu par l'algorithme dirigé par la complétude et la minimalité des prédicats [OV91] (voir Annexe 1). Si nous combinons par exemple, deux minterms primitifs $m1 : (p1 \wedge p3 \wedge p5)$ et $m2 : (p1 \wedge p3 \wedge p6)$ par l'opérateur logique OU, nous obtenons le minterm suivant :

$(p1 \wedge p3 \wedge p5) \vee (p1 \wedge p3 \wedge p6) = (p1 \wedge p3) \wedge (p6 \vee p5) = (p1 \wedge p3)$ qui définit un autre fragment potentiel. Ce minterm avec les six autres minterms ($m3, m4, m5, m6, m7, m8$) forment un schéma de fragmentation de sept fragments.

Notre objectif est de générer tous les schémas de fragmentation possibles d'une table donnée T. On peut s'interroger sur leur nombre.

A ce stade, tous les schémas de fragmentation possibles d'une table donnée peuvent être générés. Maintenant, il faut évaluer le meilleur schéma de fragmentation. Cette évaluation est assurée par le modèle de coût.

4.2 Le modèle de coût dans notre contexte

Le modèle de coût dépend de l'objectif de l'optimisation. Dans notre contexte, l'unité de mesure est le nombre d'entrées sorties. L'optimalité et la qualité des solutions obtenues par ces algorithmes sont liées à ce modèle de coût. le modèle de coût doit être précis et prendre en compte les paramètres importants déterminés par l'administrateur de la base de donnée. La sélection du schéma de fragmentation optimal : consiste à déterminer le schéma de fragmentation garantissant la meilleure performance parmi tous les schémas possibles. Afin de ré-implémenter ce module nous avons reporté les hypothèses et les paramètres déjà élaborés dans [BEL00].

4.2.1 Les hypothèses

Pour présenter le modèle de coût afin d'évaluer un ensemble de requêtes décisionnelles, il a été donné une approximation du nombre des entrées-sorties et pas prise en compte du coût de CPU et coût COM de communication sur le réseau. Ce choix est justifié dans la mesure où, dans les grandes bases de données, la contribution du coût CPU n'est pas aussi significative que le coût des entrées-sorties [FKL 97, BKL98b, YKL97, GUP99]. Notons que le modèle de coût utilisé et implémenté est celui réalisé par [BBK06] c'est un modèle qui peut facilement être étendu pour inclure les deux coûts non pris en compte. Nous considérons également que l'entrepôt est centralisé.

Soit un entrepôt de données modélisé par un schéma en étoile ayant d tables de dimension et une table des faits. Sur ce schéma, un ensemble de requêtes sont définies $Q=\{Q_1, Q_2 \dots Q_s\}$ chaque requête $Q_i (1 \leq i \leq s)$.

4.2.3 La formule du modèle de coût

Le coût est exprimé par l'équation suivante [BBK06]:

$$TIOC(Q, \phi) = \sum_{k=1}^m \sum_{i=1}^n \frac{M_j \cdot F \cdot L}{K} \cdot Pertinence(Q_k, S_i)$$

Où, $Pertinence(Q_k, S_i) = \begin{cases} 1 & \text{si le sous schéma } S_i \text{ participe dans l'évaluation de } Q_k, \\ 0 & \text{sinon.} \end{cases}$

Chaque requête Q_k est exécutée sur chaque schéma de fragmentation $FS_i\{S_1, S_2, \dots, S_{N_i}\}$. Afin d'identifier les sous schémas en étoile pertinents pour la requête Q_k ,

M_j , F , L et PS représentent le nombre de prédicats de sélection utilisés pour définir les fragments de sous schéma en étoile S_j , la cardinalité de la table des faits (nombre de tuples) F , la longueur, en octets, de chaque tuple de la table des faits et la taille d'une page disque (en octets), respectivement. Le coût total pour exécuter l'ensemble des requêtes est exprimé par l'équation suivante [BBK06]:

$$TIOC(Q, \phi) = \sum_{k=1}^m \sum_{i=1}^n \frac{M_j \cdot F \cdot L}{K}$$

Le problème de fragmentation d'un entrepôt de données relationnel consiste à partitionner un schéma en étoile, tel que le coût total d'exécution de requêtes soit minimal ou en d'autres termes, le gain soit maximisé :

Maximiser $Eval(Q, FS_i) = (TIOC(Q, \phi) - TIOC(Q, FS_i))$ tel que $N_i \leq W$, avec $TIOC(Q, \phi)$ représente le coût total d'exécution de l'ensemble de requêtes sur le schéma de l'entrepôt non fragmenté.

4.3 Algorithme proposé

Concernant l'algorithme d'évaluation des requêtes par chaque schéma, nous présentons la solution proposée sous forme d'algorithme tabou combiné avec séparation/évaluation

Algorithme

Entrée :

Maxschéma ($N = \sum_{i=1}^p m_i$: nombre de fragment de la table de dimension Di)
 Un schéma de fragmentation initial choisi aléatoirement FSI
 Liste tabou Liste-tabou; initialement vide
 W: seuil choisi par l'administrateur ($W \leq N$)

Sortie:

Un schéma optimal

Initialisation:

Liste-tabou $\leftarrow \emptyset$ /*ensemble vide
 FSmeilleur $\leftarrow$ FSI /* Initialement Meilleur schéma est FSI*/
 Nbrschema $\leftarrow 0$

Debut

Repete

calculer-cout(FSI) /* $IOC(Q_k, FS_i) = \left(\sum_{j=1}^{N_i} Pertinence(Q_k, FS_j) \times \prod_{i=1}^{M_j} Sel_F^{P_i} \left(\frac{\|F\| \times L}{PS} \right) \right)$ */

Calculer-nbr-prédicat-de-FSI(NP)

Repete

Chercher-ens-de-schémas-ayant-même-prédicat-PNP

J=1

Calculer-nbr-schémas (NBFS)

Répéter

Calculer-cout(FS(j, NP))

J=j+1

Jusqu'à j > NBFS

Liste-FSmin $\leftarrow$ FS(min, NP) /*Extraire-cout_min(FS(min, NP)) */

NP = NP-1

Jusqu'à NP=0

FSmeilleur $\leftarrow$ FSI

Pour j= 1 à NP /*Comparer-couts (FS(min,j) , FSI)*/

Si cout(FSI) > cout Liste-FSmin (FS(min,j))

Alors FSmeilleur $\leftarrow$ Liste-FSmin (FS(min,j)) /*Extraire-FS meilleur de Liste Smin*/

Fin faire pour

Label : Si FSmeilleur $\notin$ Liste tabou /* si n'existe pas*/

Alors Liste-tabou $\leftarrow$ FSmeilleur

Sinon FSmeilleur $\leftarrow$ suivant Liste-FSmin aller à label

FSI $\leftarrow$ FSmeilleur

Nbrschema = Nbrschema +1

Jusqu'à Nbrschema=(Maxschema ou W) ou Liste-FSmin = Liste-tabou

Schema-optimal $\leftarrow$ min-liste-tabou

Fin.

5 Scénario expérimenté

Dans cette étape, nous allons mettre en application la démarche proposée sur une partie de l'entrepôt de données qui est celui généré par l'outil Apb.exe, il contient quatre tables de dimension et une table des faits. Les tables sont : *Avtcars*(table des faits) , *Prodlevel* (table de dimension), *Custlevel* (table de dimension), (table de dimension), *Chanlevel* (table de dimension).

Soit le schéma en étoile suivant :

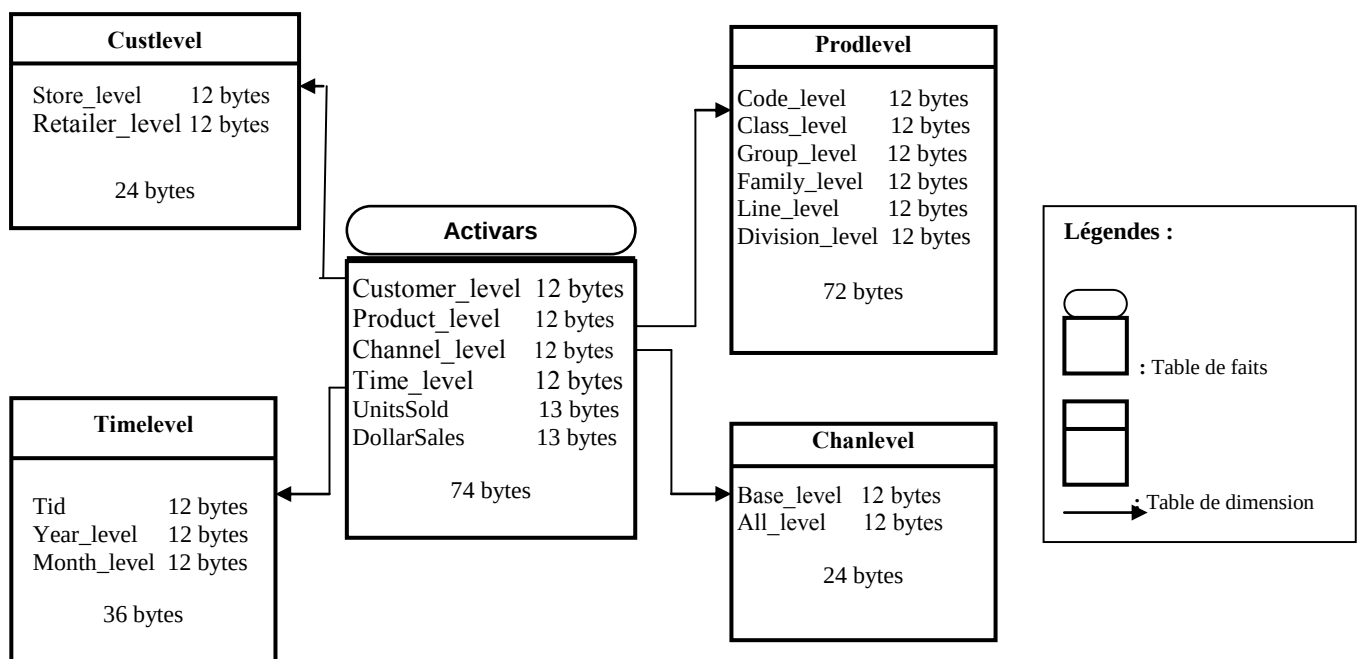


Figure 17 Schéma en étoile de l'entrepôt

- Nous avons utilisé 55 requêtes décisionnelles, chacune contenant un ensemble de prédicats de sélection. La liste des requêtes est donnée dans la table en Annexe2 avec 35 prédicats de sélection définis sur 9 attributs (Class-Level, GroupLevel, FamilyLevel, LineLevel, DivisionLevel, YearLevel, MonthLevel, RetailerLevel, AllLevel). Chaque prédicat possède un facteur de sélectivité calculé sur l'entrepôt réel. Notre algorithme a été implémenté en utilisant Builder C++ sur une machine de 1Go de RAM.

- Une fois toutes les requêtes établies, nous extrayons les prédicats intervenants dans les requêtes et les attribuant aux tables de dimension correspondantes. Le résultat est donné ci dessous :

Tables	Prédicats simples
PRODLEVEL	p_1 :Class_level= PO0HV1RICH5W p_2 :Class_level=CI493YZ9KZUJ p_3 :Class_level=FDXAQ1N5U026
	P_4 :Group_level=E4NJTW0ZR9FN
	P_5 :Family_level=BEMFVK0N8125 P_6 :Family_level=UJHZ4TZMJT6V P_7 :Family_level=SHDF8QT29KFF P_8 :Family_level=AGG214DG271Q
	P_9 :Line_level=MJ1F1U1EG009
	p_{10} :Division_level=BCR2T4K2K9D3 p_{11} :Division_level=XRLXY6H61SLC p_{12} :Division_level=RC5406URP1IE p_{13} :Division_level=G4HA5YITG3H7
TIMELEVEL	p_1 :Year_level=1995 p_2 :Year_level=1996
	P_3 :Month_level=1 P_4 :Month_level=2 P_5 :Month_level=3 P_6 :Month_level=4 P_7 :Month_level=5 P_8 :Month_level=6 P_9 :Month_level=7 P_{10} :Month_level=8 P_{11} :Month_level=9 P_{12} :Month_level=10 P_{13} :Month_level=11 P_{14} :Month_level=12
CUSTLEVEL	p_1 :Retailer_level=Z6OFS4YAAD4J p_2 :Retailer_level=RQJNEN0UPKMJ p_3 :Retailer_level=NXEYFSIQE3JM
CHANLEVEL	p_1 :All_level=ABCDEFGHJKLM p_2 :All_level=BCDEFGHIJKLM p_3 :All_level=CDEFGHIJKLMN p_4 :All_level=DEFGHIJKLMNO p_5 :All_level=EFGHIJKLMNOP

Tableau 3 L'ensemble des prédicats et les tables de dimension correspondantes

- Puis nous fragmentons les tables de dimension selon les prédicats sélectionnés comme le montre la table suivante :

Dimension	Fragments	Condition de Fragmentation
PROLEVEL	PRODL1 PRODL2 PRODL3 PRODL4 PRODL5 PRODL6 PRODL7 PRODL8 PRODL9 PRODL10 PRODL11 PRODL12 PRODL13	Class_level = PO0HV1RICH5W Class_level =CI493YZ9KZUJ Class_level =FDXAQ1N5U026 Group_level =E4NJTW0ZR9FN Family_level =BEMFVK0N8125 Family_level =UJHZ4TZMJT6V Family_level =SHDF8QT29KFF Family_level =AGG214DG271Q Line_level =MJ1F1U1EG009 2297584 Division_level =BCR2T4K2K9D3 Division_level =XRLXY6H61SLC Division_level =RC5406URP1IE Division_level =G4HA5YITG3H7
TIMELEVEL	TIMEL1 TIMEL2 TIMEL3 TIMEL4 TIMEL5 TIMEL6 TIMEL7 TIMEL8 TIMEL9 TIMEL10 TIMEL11 TIMEL12 TIMEL13 TIMEL14	Year_level =1995 Year_level =1996 Month_level =1 Month_level =2 Month_level =3 Month_level =4 Month_level =5 Month_level =6 Month_level =7 Month_level =8 Month_level =9 Month_level =10 Month_level =11 Month_level =12
CUSTLEVEL	CUSTL1 CUSTL2 CUSTL3	Retailer_level =Z6OFS4YAAD4J Retailer_level =RQJNEN0UPKMQ Retailer_level =NXEYFSIQE3JM
CHANLEVEL	CHANL1 CHANL2 CHANL3 CHANL4 CHANL5	All_level =ABCDEFGHJKLM All_level =BCDEFGHIJKLM All_level =CDEFGHIJKLMN All_level =DEFGHIJKLMNO All_level =EFGHIJKLMNOP

Tableau 4 les fragments des tables de dimension

- On génère tous les schémas de fragmentation (2730 schémas):

Fragments	Clause
Actvars_1	ACTVARS $\times$ PRODL1 $\times$ TIMEL1 $\times$ CUSTL1 $\times$ CHANL1
Actvars_2	ACTVARS $\times$ PRODL1 $\times$ TIMEL1 $\times$ CUSTL1 $\times$ CHANL2
Actvars_3	ACTVARS $\times$ PRODL1 $\times$ TIMEL1 $\times$ CUSTL1 $\times$ CHANL3
Actvars_4	ACTVARS $\times$ PRODL1 $\times$ TIMEL1 $\times$ CUSTL1 $\times$ CHANL4
Actvars_5	ACTVARS $\times$ PRODL2 $\times$ TIMEL1 $\times$ CUSTL1 $\times$ CHANL5
Actvars_6	ACTVARS $\times$ PRODL2 $\times$ TIMEL1 $\times$ CUSTL2 $\times$ CHANL1
Actvars_7	ACTVARS $\times$ PRODL2 $\times$ TIMEL1 $\times$ CUSTL2 $\times$ CHANL2
Actvars_8	ACTVARS $\times$ PRODL2 $\times$ TIMEL1 $\times$ CUSTL2 $\times$ CHANL3
.....	
Actvars_2730	ACTVARS $\times$ PRODL13 $\times$ TIMEL14 $\times$ CUSTL3 $\times$ CHANL5

Tableau 5 Les fragments de la table des faits

- Dans ce qui suit, on va introduire l'algorithme proposé passant par ses étapes énumérées (Rappelons qu'à l'entrée de notre algorithme, on aura besoin de l'ensemble des schémas de fragmentation pour choisir un schéma aléatoire de cet ensemble) :

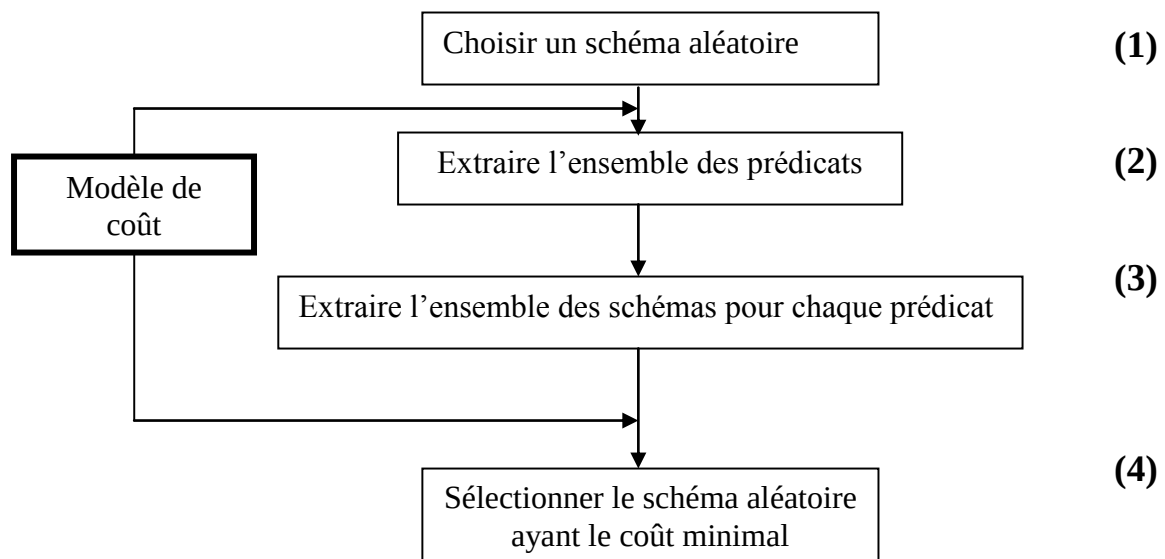


Figure 18 Les étapes de notre algorithme proposé

- 1) Choisir un schéma aléatoire : le choix du schéma à l'entrée se fait d'une manière aléatoire de l'ensemble des schémas générés de l'étape précédente. Dans le premier schéma de cette première expérimentation du tableau (référence), le schéma est le suivant avec son coût calculé par le modèle de coût :

Fragments	Clause
Actvars_1	ACTVARS $\times$ PRODL1 $\times$ TIMEL2 $\times$ CUSTL1 $\times$ CHANL1

- 2) Dans cette étape, nous devons extraire l'ensemble des prédicats relatifs à notre schéma sélectionné. Les prédicats sont :

Prédicats simples
Class_level="PO0HV1RICH5W"
Year_level=1995
Retailer_level="Z6OFS4YAAD4J"
All_level="ABCDEFGHijkl"

- 3) Pour chaque prédicat, extraire l'ensemble des schémas et leurs coûts correspondants.

NB : on doit s'assurer que les schémas extraits ne soient pas redondants,(ne figurent pas déjà).

- 4) On choisira le schéma dont le coût est minimal puis on réitère jusqu'à ce qu'on passe par tous les schémas (la liste tabou contient tous les schémas engendrés)

6 Discussion des résultats

A titre comparatif nous avons implémenté deux heuristiques :l'algorithme génétique [BBK05] (Annexe 4) et Tabou Séparation / Évaluation.

La Figure 19 montre l'évolution du nombre d'E/S par rapport au nombre d'attributs pour l'algorithme Tabou et l'algorithme génétique : Nous constatons Quand le nombre d'attributs diminue, le nombre de prédicats diminue sachant que le nombre de prédicats utilisés par les requêtes a un effet considérable sur la performance des requêtes. Cependant

lorsque ce nombre est petit, la plupart des requêtes ne bénéficient pas de la fragmentation. Ceci est expliqué par le fait qu'elles accèdent à un nombre important de sous schémas, voire la totalité des sous-schémas si elles ne possèdent pas des prédicats définis sur des attributs de fragmentation.

Par conséquent, plusieurs opérations d'union sont nécessaires pour avoir le résultat final. Par contre si le nombre de prédicats est important, le nombre de sous schémas valides pour chaque requête est réduit (surtout pour celles n'utilisant que des attributs de fragmentation) ce qui implique le chargement de moins de données (les sous schémas valides seulement). Les résultats se rapprochent. Toutefois lorsque le nombre d'attributs augmente le nombre E/S diminue.

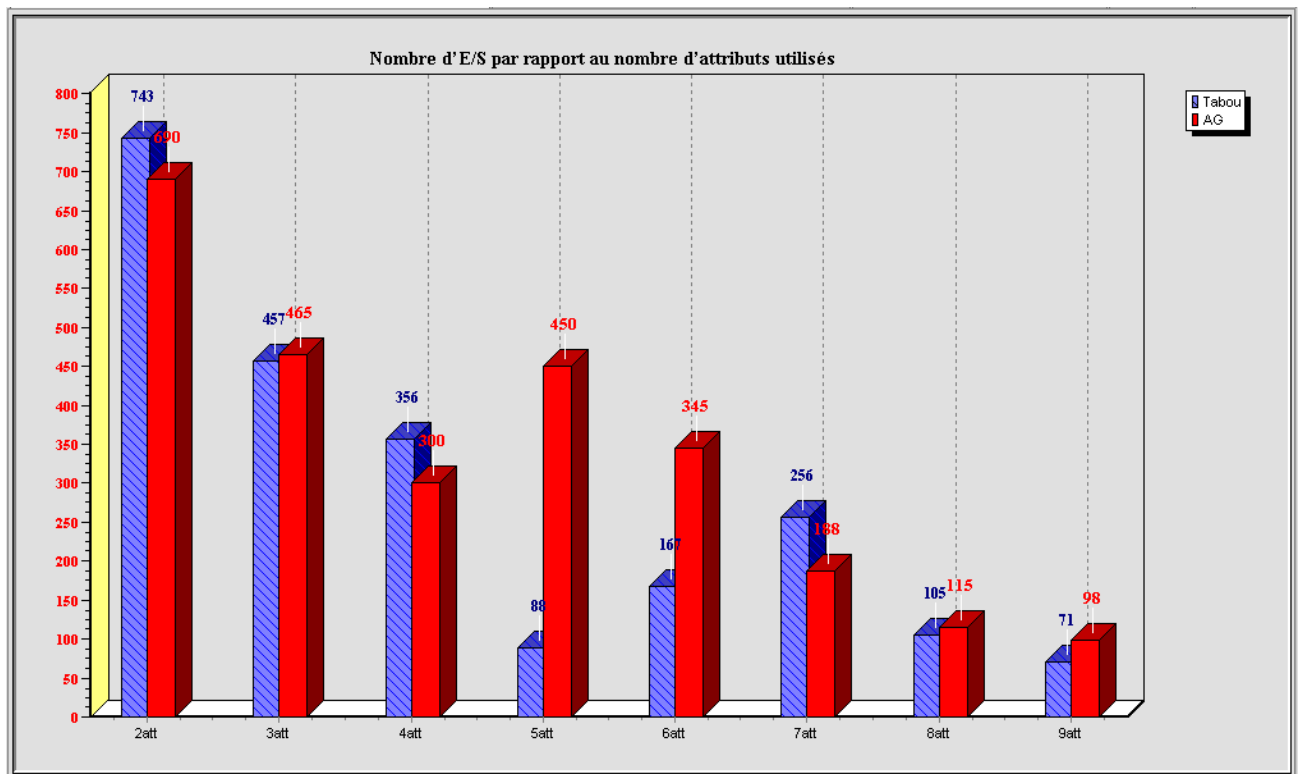


Figure 19 Nombre d'E/S par rapport au nombre d'attributs utilisées

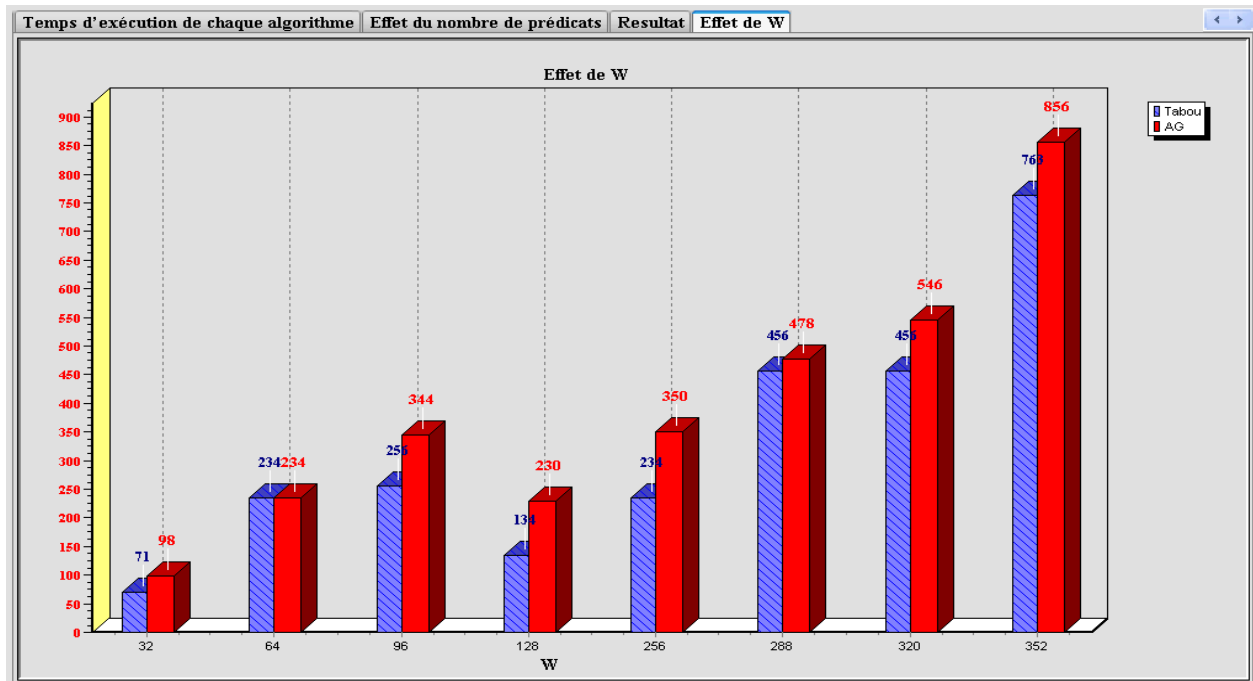


Figure 20 Effet du seuil W

La Figure 20 représente l'évolution du nombre d'E/S par rapport au seuil. On constate qu'un seuil de 32 est le plus adapté il représente 10% du nombre totale de fragments.

L'augmentation du seuil améliore généralement les performances des requêtes car en relâchant W, plus d'attributs sont utilisés pour fragmenter l'entrepôt. Lorsque W est grand les domaines sont décomposés en plus de partitions et donc chaque partition est moins volumineuse. Cela implique moins de données chargées pour exécuter les requêtes utilisant les attributs de fragmentation.

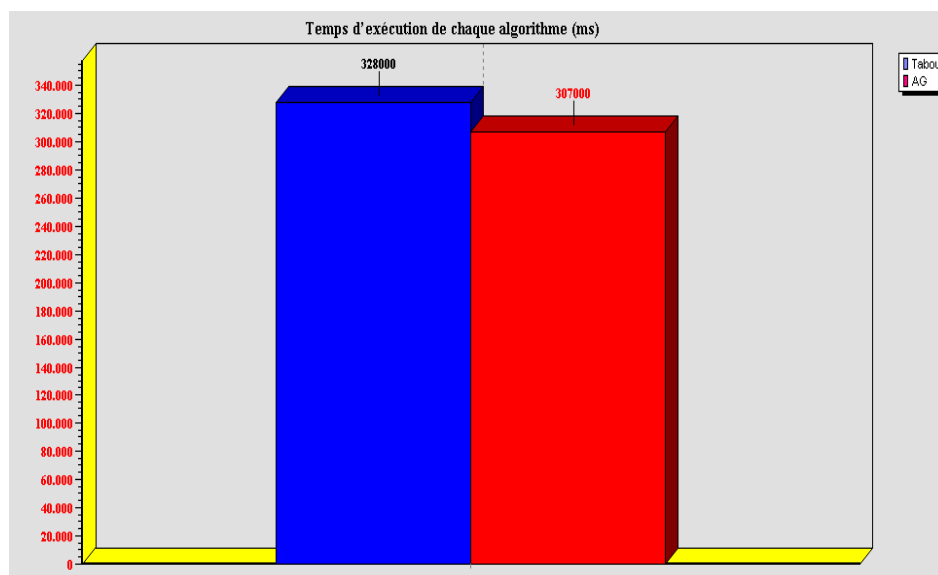


Figure 21 Temps d'exécution de chaque algorithme

La figure 21 montre le temps moyen d'exécution de chaque algorithme. Tabou/Séparation est moins performant par le fait qu'il est confronté à une recherche plus fine dans des optimums locaux donc il consomme plus de temps d'exécution. L'algorithme génétique est légèrement plus rapide.

Si l'administrateur privilégie la qualité de la solution, il choisira Tabou, sinon le génétique s'il privilégie la rapidité d'exécution de son algorithme de sélection.

7 Conclusion

Dans notre étude nous avons développé un algorithme pour la sélection d'un schéma de fragmentation optimal, cet algorithme se base sur l'algorithme tabou combiné avec séparation/évaluation

Pour évaluer notre algorithme, nous avons fait une étude comparative avec l'algorithme génétique [BBK05] suivi par une discussion sur les remarques faites après l'expérimentation Cette comparaison est réalisée avec une étude expérimentale basée sur un modèle de coût calculant le nombre d'entrées/sorties nécessaire pour exécuter un ensemble de requêtes.

Notre algorithme se base sur une recherche locale afin de déterminer un schéma optimal. Par contre l'algorithme génétique fait sa recherche sur un espace plus grand. De ce fait le nombre E/S par rapport au nombre d'attribut est plus faible dans tabou et par rapport à la variation de w , le nombre E/S augmente proportionnellement avec w mais moins que dans génétique.

Nous concluons que la sélection d'un schéma de fragmentation optimal est mieux adaptée avec Tabou qu'avec génétique toutefois génétique l'emporte en terme de temps d'exécution.

Conclusion Générale

Les entrepôts de données ont été abordés par les industriels en premier lieu, par la suite les chercheurs se sont penchés sur ce concept. Afin d'atteindre un niveau de performance acceptable, les deux communautés ont développé des techniques d'optimisation des requêtes au niveau de chaque phase de la conception d'un entrepôt (phase conceptuelle, phase logique, phase physique).

Dans le domaine des entrepôts de données, nous nous sommes plus particulièrement intéressés au problème de la performance. En se basant sur l'intuition que des connaissances sur les données entreposées et leur usage peuvent contribuer à optimiser leur accès, les solutions d'indexation et de matérialisation de vues font appel à des techniques de fouille de données. Ceci tend vers l'automatisation des tâches d'optimisation de performance de requêtes, qui constituent une part importante du travail d'administration d'un entrepôt.

Dans le cadre de notre travail, et afin de réduire le coût d'exécution des requêtes, il était indispensable de recourir à la fragmentation horizontale. Nous nous sommes attachés à suivre une méthodologie de fragmentation horizontale de la table des faits et des tables de dimension. Nous avons illustré son déroulement à travers un exemple. Il est apparu qu'il ne faut pas fragmenter directement la table de faits (en général la plus volumineuse), mais fragmenter plutôt au premier lieu les tables de dimension puis la table de faits en utilisant leur schéma de fragmentation obtenu. Les performances de la fragmentation sont évaluées par l'intermédiaire d'un modèle de coût calculant le nombre d'entrées sorties nécessaire à l'exécution d'un ensemble de requêtes.

A cet effet, nous avons proposé et mis-en-œuvre l'algorithme Tabou combiné avec séparation et évaluation. L'algorithme tabou s'avère efficace pour l'optimisation des problèmes avec un très grand espace de recherche. Séparation/évaluation est appliqué sur un

critère de recherche afin d'éliminer les branches dont les solutions de mauvaises fitness (fonction objectif supérieure). Cette fonction est caractérisée par un modèle de coût qui évalue le coût d'exécution d'un ensemble de requêtes fréquentes exécutées sur un entrepôt de données relationnel modélisé par un schéma en étoile.

Afin de mettre en valeur notre approche, les résultats des tests obtenus ont été comparés avec ceux d'algorithme génétique. Du fait que notre algorithme se concentre sur une recherche locale afin de déterminer un schéma optimal, l'algorithme génétique fait sa recherche sur un espace plus grand. De ce fait le nombre E/S par rapport au nombre d'attribut est plus faible dans tabou. Nous constatons de plus que lorsque le nombre d'attributs augmente le nombre E/S diminue.

En faisant varier W et en utilisant les mêmes prédicats, Tabou donne de meilleurs résultats pour toutes les valeurs (variation) de W . L'algorithme Génétique quant à lui est moins performant ceci est dû à son espace de recherche qui est plus grand. Il est normal de constater que lorsque W augmente on considère plus de schéma donc plus d'E/S. Alors la sélection d'un schéma de fragmentation optimal est mieux adaptée avec Tabou

Toutefois Tabou consomme plus de temps d'exécution. L'algorithme génétique est légèrement plus rapide.

Enfin le choix revient à l'administrateur de l'entrepôt. S'il privilégie la qualité de la solution, il choisira Tabou, sinon, le génétique, s'il privilégie la rapidité d'exécution de son algorithme de sélection.

En termes de perspectives nous proposons ce qui suit:

1. Explorer la fragmentation verticale. Nous proposons d'adapter l'algorithme proposé. Une adaptation directe consiste à remplacer les sous domaines par les attributs de tables à fragmenter.
2. Proposer une architecture des schémas en flocons de neige pour faciliter l'usage de la fragmentation ainsi réduire le coût des requêtes.
3. Développer des algorithmes qui tiennent compte de l'évolution des requêtes tant dans leurs structures que dans leurs fréquences, d'où une éventuelle fragmentation dynamique.

Annexes

1 Infrastructure système

Tout système informatique repose sur une combinaison de ressources matérielles et logicielles. Pour un entrepôt de données, il faut en général prévoir des serveurs puissants avec une grande capacité de stockage de données. Plusieurs logiciels de base de données supportent maintenant les besoins des entrepôts de données, et il y a de nombreux fournisseurs de solutions spécialisées. Un autre aspect de l'infrastructure système est le transfert des données. Les données doivent pouvoir être acheminées en un temps acceptable entre les systèmes de production et l'entrepôt (acquisition des données), mais il faut aussi prévoir une bande passante suffisante pour les distribuer (distribution des données) et permettre un accès en temps réel aux utilisateurs. Tous ces points doivent être considérés, et une faiblesse dans un seul de ces aspects peut nuire grandement au projet d'entrepôt de données.

2 Les méta-données

Les méta-données représentent la forme la plus utile de documentation de l'entrepôt. Ce sont des éléments (sous forme de documents PDF en ligne, pages Web, aide contextuelle, etc.) qui servent à expliquer le fonctionnement des données et l'état de l'entrepôt. L'utilisation de méta-données permet à la fois aux utilisateurs et à l'équipe d'entretien de l'entrepôt de se retrouver plus facilement. Les méta-données existent sous deux formes: contrôle et utilisateur [ONB97]. Les méta-données de contrôle sont utilisées pour veiller au bon fonctionnement de l'entrepôt. Quelques exemples de ces données sont : une liste des données qui ont causé des problèmes durant le chargement, l'information sur la taille des tables, le contenu des tables et des vues, etc. Les méta données orientées utilisateur servent à guider l'utilisateur final dans l'entrepôt. C'est une documentation qui définit les termes clairement, sans ambiguïté, et qui décrit les données accessibles à l'utilisateur. Par exemple, ces méta-données peuvent expliquer comment faire la somme des ventes en fonction des années dans le logiciel mis à la disposition des utilisateurs.

Le rôle des méta-données dans un entrepôt ne doit pas être sous-estimé. Pour avoir un entrepôt facile à entretenir pour les programmeurs et compréhensible pour l'utilisateur final, il faut disposer d'outils simples et de structures bien définies. L'utilisateur doit avoir accès aux informations nécessaires pour se retrouver dans l'entrepôt et bien interpréter les données. C'est pourquoi les méta-données doivent être rédigées dans des termes faciles à

comprendre pour tous les types d'utilisateurs. Elles devraient lui permettre de répondre aux questions suivantes au sujet de l'entrepôt et des magasins de données

1. Quels sont les sujets traités dans l'entrepôt et les magasins de données ?
2. Quels faits et dimensions sont inclus dans l'entrepôt et les magasins de données ?
Quelle granularité ont les tables de faits ?
3. Comment les données du magasin sont-elles dérivées des données de l'entrepôt ?
Quelles sont les règles (transformations) appliquées ?
4. Comment les données de l'entrepôt sont-elles dérivées des données sources ? Quelles sont les règles (transformations) appliquées ?
5. Quels rapports et requêtes prédéfinies sont disponibles pour visualiser les données ?
6. Quels outils et techniques d'analyse sont disponibles ?
7. Qui est responsable de la qualité des données, et à qui doit-on faire les demandes de changement ?

Mais il est de plus en plus recommandé d'utiliser les méta-données pour gérer le chargement des données, ce qui rend l'entrepôt beaucoup plus flexible et facile à modifier. Des outils de chargement et d'entretien d'entrepôts commencent à apparaître [RBN02]. Les méta-données deviennent alors un moteur pour la création des requêtes dans l'entrepôt, à la fois pour l'entretien et pour l'utilisation des données.

3 La découverte des données

La phase de découverte des données est généralement celle qui prend le plus de temps à l'équipe de développement [ONB97]. De nombreux intervenants spécialistes sont nécessaires (par exemple les programmeurs qui ont conçu les systèmes qui serviront de source de données, des analystes, des personnes impliquées dans la gestion de l'entreprise, etc.). Il faut parcourir les différentes sources de données (bases de données dans l'entreprise) pour trouver les données d'intérêt qui seront chargées dans l'entrepôt.

En temps normal, les données de ces systèmes sont incomplètes et doivent subir un nettoyage et des transformations avant d'être utilisables par l'entrepôt. Il y a normalement des incohérences entre les différents systèmes, et pour intégrer correctement les données, il faut réussir à réunir (joindre) les différentes bases de données entre elles. Il peut arriver que des champs soient manquants (données manquantes), que d'autres aient été incorrectement saisies (par exemple, les noms des clients peuvent être saisis différemment ou

avoir des erreurs entre les systèmes). C'est pourquoi ce travail est très long, une bonne partie des données peut être traitée automatiquement. Les manipulations nécessaires pour rendre l'ensemble des données cohérentes sont parmi les plus importantes pour s'assurer que les résultats obtenus à partir de l'entrepôt seront justes.

4 L'acquisition des données

Une fois les données identifiées, il reste à remplir l'entrepôt. Un processus d'extraction, de transformation et de chargement des données est alors préparé (processus *ETL*, *Extract Transform and Load*). L'étape d'extraction des données se fait généralement directement sur les systèmes de production, dans les bases de données en créant un fichier de données. Ce fichier sera par la suite téléchargé sur un deuxième système qui se chargera du reste des manipulations de données. Puisque cette étape se fait sur les systèmes de production, il arrive souvent qu'une fenêtre de temps très limitée soit disponible pour effectuer le travail d'extraction, par exemple durant la nuit de 2 à 3 heures du matin. Parfois, il est même impossible d'arrêter ces systèmes. Une méthode doit alors être trouvée pour s'assurer de l'intégrité des données extraites, tout en dérangeant le moins possibles les logiciels en utilisation sur les serveurs.

Les transformations sont utilisées pour nettoyer les données et pour créer les clés qui serviront dans l'entrepôt. Il peut arriver que des données concernant la même entité (personne, entreprise, etc.) soient présentes dans différents systèmes, mais qu'il n'y ait pas de façon de joindre ces données automatiquement. Par exemple, un des systèmes peut identifier des personnes avec leur numéro d'assurance sociale, et un autre avec un code arbitraire à partir de leur nom et de leur date de naissance. Afin de pouvoir joindre les données et de les insérer dans l'entrepôt, il est nécessaire de créer des clés supplémentaires pour chaque enregistrement (comme un nombre), ce qui permet de les identifier uniquement et de les joindre correctement. Ces clés substituts deviennent alors les clés utilisées pour les jointures dans l'entrepôt.

Il existe des logiciels qui sont spécialisés dans le processus *ETL* d'un entrepôt de données. Il est souvent souhaitable de faire appel à un utilitaire existant sur le marché pour éviter des frais de développement et de test. Certains auteurs disent que peu importe notre situation, elle n'est jamais assez unique pour justifier le développement à l'interne de la majorité des logiciels nécessaires dans un entrepôt de données, y compris les utilitaires *ETL* [ONB97]. Cependant, une recherche rapide montre qu'il y en a des centaines. Aussi, en général ils ne produisent pas un code optimal, ils sont souvent plus coûteux que le temps de développement qu'ils remplacent et leur surabondance rend difficile le processus de sélection [SCB003].

5 La distribution des données

Une fois la phase de chargement des données terminée, il faut les distribuer dans tous les centres d'analyse. Selon les besoins des utilisateurs, il est possible de concevoir un entrepôt de données à l'aide d'un ou de plusieurs magasins de données (*data marts*). Il est possible de réaliser un entrepôt de données où tout est centralisé et où il n'y a pas de magasins de données. Cependant, il est parfois avantageux d'en utiliser, que ce soit pour améliorer le temps de réponse lors de l'exécution de certaines analyses, ou en distribuant physiquement les magasins pour réduire la distance que les données ont à parcourir pour se rendre à l'utilisateur, etc.

Il existe six sortes de magasins de données [ONB97] :

- ☒ Satellite : récupère toutes ses données de l'entrepôt;
- ☒ Alimenteur (*feeders*) : alimente l'entrepôt;
- ☒ Partition : est un constituant d'un entrepôt virtuel partitionné;
- ☒ Mini-entrepôt : comme un entrepôt, sans aller jusqu'au bout d'un projet complet d'entrepôt dans une entreprise (peut être vu comme un projet-pilote);
- ☒ Indépendant : c'est un mini-entrepôt avec des programmes de chargement de données qui sont implémentés par des départements au sein d'une entreprise, sans avoir de lien avec un entrepôt de données central;
- ☒ Mixé : représente une architecture d'entrepôts de données où plusieurs sortes de magasins sont utilisées.

En fonction du type d'architecture retenue, la phase de distribution des données peut continuer même une fois que l'entrepôt principal est prêt. Tous les magasins doivent éventuellement être mis à jour à partir du contenu de l'entrepôt.

6 Les logiciels d'analyse

Une fois les données dans l'entrepôt, l'exploitation devient possible avec de nouvelles méthodes. Les requêtes ad hoc sont possibles, mais des outils plus spécialisés comme des logiciels *OLAP* et de forage de données donnent à l'analyste ou tout autre utilisateur beaucoup plus de puissance et de facilité pour l'accès à toutes les ressources de l'entrepôt (données et méta données). Les logiciels *OLAP* existent sous plusieurs formes. Le modèle dimensionnel est une forme plus naturelle de modélisation des données pour un système d'information, et ces logiciels profitent de cette propriété. Le but de ces logiciels est de permettre à un utilisateur un accès simple aux données sous forme de fenêtres et de graphiques. Il y a des logiciels *MOLAP* (*multi-dimensional OLAP*) qui utilisent une base de données dimensionnelle (parfois propriétaire) et les logiciels *ROLAP* (*relational OLAP*) qui utilisent une base de données relationnelle (Oracle, Access, DB2, etc.). Il y a aussi des logiciels qui importent les données, peu importe leur format (par exemple, un fichier texte). Ces derniers sont qualifiés de *OLAP Client* parce qu'ils fonctionnent comme une application sur un ordinateur personnel.

COM_MIN Algorithme gérant des prédicats minimaux et complets

- 1: **Données** : Une relation R et un ensemble de prédicats simples P définis sur R
- 2: **Sortie** : Un ensemble complet et minimal de prédicats simple P' de P
- 3: **Déclaration** : F : l'ensemble de fragments. f_i : un fragment défini par un miniterm m_i
- 4: **Règle 1** : Toute relation est partitionnée en au moins deux fragments qui seront accédés différemment par au moins une application.
- 5: Initialisation :
- 6: $P' = \emptyset$
- 7: Chercher un prédicat q_i de P tel que q_i partitionne R en respectant la règle 1
- 8: $P' = q_i$; $P = P - q_i$; $F = f_i$
- 9: **repeat**
- 10: Chercher un prédicat q_i de P tel que q_i partitionne f_k de F en respectant la règle 1
- 11: $P' = P' \cup q_i$; $P = P - q_i$; $F = F \cup f_i$
- 12: **If** il existe $q_i \in P'$ qui n'est pas pertinent **then**
- 13: $P' = P' - q_i$
- 14: $F = F - f$
- 15: **end if**
- 16 : **until** P' est complet

ID	Formulation en SQL
1	<pre>select Time_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.Class_LEVEL='PO0HV1RICH5W' group by Time_level</pre>
2	<pre>select Time_level,sum(Dollarcost) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.Class_LEVEL='CI493YZ9KZUJ' group by Time_level</pre>
3	<pre>select Time_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.Class_LEVEL='FDXAQ1N5U026' group by Time_level</pre>
4	<pre>select Time_level,Avg(UNITSSOLD) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.Class_LEVEL not in('PO0HV1RICH5W', 'CI493YZ9KZUJ', 'FDXAQ1N5U026') group by Time_level</pre>
5	<pre>select Time_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.group_LEVEL='E4NJTW0ZR9FN' group by Time_level</pre>
6	<pre>select Time_level,Max(UNITSSOLD) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.group_LEVEL<>'E4NJTW0ZR9FN' group by Time_level</pre>
7	<pre>select Time_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.family_LEVEL='BEMFVK0N8125' group by Time_level</pre>
8	<pre>select Time_level,sum(Dollarcost) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.family_LEVEL='UJHZ4TZMJT6V' group by Time_level</pre>
9	<pre>select Customer_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.family_LEVEL='SHDF8QT29KFF' group by customer_level</pre>

10	<pre> select Customer_level,Avg(Unitsold) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.family_LEVEL='AGG214DG271Q' group by Customer_level </pre>
11	<pre> select Channel_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.family_LEVEL not in('BEMFVK0N8125','UJHZ4TZMJT6V','SHDF8QT29KFF', 'AGG214DG271Q') group by Channel_level </pre>
12	<pre> select Product_level,Sum(Dollarcost) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.LINE_LEVEL = 'MJ1F1U1EG009' group by Product_level </pre>
13	<pre> select Product_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.LINE_LEVEL <> 'MJ1F1U1EG009' group by Product_level </pre>
14:	<pre> select Customer_level,Avg(unitsold) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.DIVISION_LEVEL = 'BCR2T4K2K9D3' group by Customer_level </pre>
15	<pre> select Time_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.DIVISION_LEVEL = 'XRLXY6H61SLC' group by Time_level </pre>
16	<pre> select Customer_level,Sum(Dollarcost) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.DIVISION_LEVEL = 'RC5406URP1IE' group by Customer_level </pre>
17	<pre> select Channel_level,count(*) from ACTVARS A,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and P.DIVISION_LEVEL = 'G4HA5YITG3H7' group by Channel_level </pre>
18	<pre> select Product_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.year_LEVEL = '1995' group by Product_level </pre>

19	select Product_level,Sum(Dollarcost) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.year_LEVEL = '1996' group by Product_level
20	select Product_level,Avg(Unitsold) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '1' group by Product_level
21	select channel_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '2' group by Channel_level
22	select Product_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '3' group by Product_level
23	select Product_level,Sum(Dollarcost) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '4' group by Product_level
24	select Channel_level,Avg(Unitsold) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '5' group by Channel_level
25	select Product_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '6' group by Product_level
26	select Product_level,Sum(Dollarcost) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '7' group by Product_level

27	select Customer_level,Sum(Dollarcost) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '8' group by Customer_level
28	select Customer_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '9' group by Customer_level
29	select Customer_level,Sum(Dollarcost) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '10' group by Customer_level
30	select Customer_level,count(*) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '11' group by Customer_level
31	select Product_level,Time_level,Avg(unitssold) from ACTVARS A,TIMELEVEL T where A.TIME_LEVEL=T.TID and T.month_LEVEL = '12' group by Product_level,Time_level
32	select Product_level,Customer_level, Max(unitssold) from ACTVARS A,CUSTLEVEL C where A.CUSTOMER_LEVEL=C.STORE_LEVEL and C.RETAILER_LEVEL = 'Z6OFS4YAAD4J' group by Product_level,Customer_level
33	select Product_level,Channel_level, count(*) from ACTVARS A,CUSTLEVEL C where A.CUSTOMER_LEVEL=C.STORE_LEVEL and C.RETAILER_LEVEL = 'RQJNEN0UPKMQ' group by Product_level,Channel_level
34	select Product_level,Time_level, Min(unitssold) from ACTVARS A,CUSTLEVEL C where A.CUSTOMER_LEVEL=C.STORE_LEVEL and C.RETAILER_LEVEL = 'NXEYFSIQE3JM' group by Product_level,Time_level
35	select Product_level,Sum(Dollarcost) from ACTVARS A,CUSTLEVEL C where A.CUSTOMER_LEVEL=C.STORE_LEVEL and C.RETAILER_LEVEL not in ('Z6OFS4YAAD4J', 'RQJNEN0UPKMQ','NXEYFSIQE3JM') group by Product_level

36	select Product_level,Time_level, count(*) from ACTVARS A,CHANLEVEL CH where A.CHANNEL_LEVEL=CH.BASE_LEVEL and CH.ALL_LEVEL ='ABCDEFGHijkl' group by Product_level, Time_level
37	select Channel_level,Time_level, Sum(Dollarcost) from ACTVARS A,CHANLEVEL CH where A.CHANNEL_LEVEL=CH.BASE_LEVEL and CH.ALL_LEVEL ='BCDEFGHIJKLM' group by Channel_level, Time_level
38	select Customer_level,Channel_level, Time_level, Avg(Unitssold) from ACTVARS A,CHANLEVEL CH where A.CHANNEL_LEVEL=CH.BASE_LEVEL and CH.ALL_LEVEL ='CDEFGHIJKLMN' group by Customer_level,Channel_level, Time_level
39	select Product_level,Channel_level, Time_level,count(*) from ACTVARS A,CHANLEVEL CH where A.CHANNEL_LEVEL=CH.BASE_LEVEL and CH.ALL_LEVEL ='DEFGHIJKLMNO' group by Product_level,Channel_level, Time_level
40	select Time_level, count(*) from ACTVARS A,CHANLEVEL CH where A.CHANNEL_LEVEL=CH.BASE_LEVEL and CH.ALL_LEVEL ='EFGHIJKLMNOP' group by Time_level
41	select Customer_Level,Time_level, sum(dollarcost) from ACTVARS A,CUSTLEVEL C,PRODLEVEL P where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and P.CLASS_LEVEL='CI493YZ9KZUJ' and C.RETAILER_LEVEL='RQJNEN0UPKMQ' group by Customer_level,Time_level
42	select Customer_Level, Channel_level, Time_level, sum(dollarcost) from ACTVARS A, PRODLEVEL P,TIMELEVEL T where A.PRODUCT_LEVEL=P.CODE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1996' and P.CLASS_LEVEL='FDXAQ1N5U026' group by Customer_level, Channel_level, Time_level
43	select Customer_Level, Time_level, Avg(Unitssold) from ACTVARS A, CHANLEVEL H,PRODLEVEL P where A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and P.CLASS_LEVEL='FDXAQ1N5U026' and H.ALL_LEVEL ='ABCDEFGHijkl' group by Customer_level, Time_level

44	select Customer_Level, Time_level, Max(Unitsold) from ACTVARS A, CUSTLEVEL C, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='1' and C.RETAILER_LEVEL not in('Z6OFS4YAAD4J','RQJNEN0UPKMQ', 'NXYFSIQE3JM') group by Customer_level, Time_level
45	select Customer_Level, sum(dollarcost), Avg(Unitsold) from ACTVARS A, CUSTLEVEL C, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='12' and C.RETAILER ='RQJNEN0UPKMQ' group by Customer_level
46	select Customer_Level, Time_level, Min(unitsold) from ACTVARS A, CHANLEVEL H, TIMELEVEL T where A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1996' and H.ALL_LEVEL='DEFGHIJKLMNO' group by Customer_level, Time_level
47	select Customer_Level, Product_level, Time_level, Sum(Dollarcost) from ACTVARS A, CUSTLEVEL C, PRODLEVEL P, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1996' and C.RETAILER_LEVEL ='NXYFSIQE3JM' and P.LINE_LEVEL='MJ1F1U1EG009' group by Customer_level, Product_level, Time_level
48	select Customer_Level, Product_level, Channel_level, Avg(unitsold) from ACTVARS A, CHANLEVEL H, PRODLEVEL P, TIMELEVEL T where A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='1' and P.DIVISION_LEVEL='XRLXY6H61SLC' and H.ALL_LEVEL='BCDEFGHIJKLM' group by Customer_level, Product_level, Channel_level
49	select Customer_Level, Product_level, Channel_level, sum(dollarcost) from ACTVARS A, CHANLEVEL H, CUSTLEVEL C, PRODLEVEL P where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and P.FAMILY_LEVEL='AGG214DG271Q' and C.RETAILER_LEVEL='NXYFSIQE3JM' and H.ALL_LEVEL='DEFGHIJKLMNO' group by Customer_level, Product_level, Channel_level

50	select Customer_Level, Time_level, Product_level, Max(Units sold) from ACTVARS A, CHANLEVEL H, CUSTLEVEL C, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='4' and C.RETAILER_LEVEL='NXEYFSIQE3JM' and H.ALL_LEVEL='BCDEFGHIJKLM' group by Customer_level, Time_level, Product_level
51	select Customer_Level, Time_level, Product_level, Channel_level, Min(Units sold) from ACTVARS A, CHANLEVEL H, CUSTLEVEL C, PRODLEVEL P, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='7' and P.GROUP_LEVEL<>'E4NJTW0ZR9FN' and C.RETAILER_LEVEL='Z6OFS4YAAD4J' and H.ALL_LEVEL='EFGHIJKLMNOP' group by Customer_level, Time_level, Product_level, Channel_level
52	select Customer_Level, Channel_level, sum(dollarcost) from ACTVARS A, CHANLEVEL H, CUSTLEVEL C, PRODLEVEL P, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.MONTH_LEVEL='11' and P.CLASS_LEVEL NOT IN ('PO0HV1RICH5W', 'CI493YZ9KZUJ', 'FDXAQ1N5U026') and C.RETAILER_LEVEL not in ('Z6OFS4YAAD4J', 'RQJNEN0UPKMQ', 'NXEYFSIQE3JM') and H.ALL_LEVEL='DEFGHIJKLMNO' group by Customer_level, Channel_level
53	select Customer_Level, Product_level, Time_level, Channel_level, Avg(Units sold) from ACTVARS A, CHANLEVEL H, CUSTLEVEL C, PRODLEVEL P, TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1995' and P.GROUP_LEVEL<>'E4NJTW0ZR9FN' and C.RETAILER_LEVEL='RQJNEN0UPKMQ' and H.ALL_LEVEL='BCDEFGHIJKLM' group by Customer_level, Product_level, Time_level, Channel_level

54	select Customer_Level, Product_level, Time_level, Channel_level, Min(UnitsSold) from ACTVARS A, CHANLEVEL H,CUSTLEVEL C,PRODLEVEL P,TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1995' and T.MONTH_LEVEL in('2','3','4') and P.CLASS_LEVEL='CI493YZ9KZUJ' and P.GROUP_LEVEL='E4NJTW0ZR9FN' and P.FAMILY_LEVEL='AGG214DG271Q' and P.LINE_LEVEL <> 'MJ1F1U1EG009' and C.RETAILER_LEVEL in ('Z6OFS4YAAD4J','RQJNEN0UPKMQ') and H.ALL_LEVEL ='ABCDEFGHIJKL' group by Customer_level, Product_level, Time_level, Channel_level
55	select Customer_Level, Product_level, Time_level, Channel_level, Max(UnitsSold) from ACTVARS A, CHANLEVEL H,CUSTLEVEL C,PRODLEVEL P,TIMELEVEL T where A.CUSTOMER_LEVEL=C.STORE_LEVEL and A.PRODUCT_LEVEL=P.CODE_LEVEL and A.CHANNEL_LEVEL=H.BASE_LEVEL and A.TIME_LEVEL=T.TID and T.YEAR_LEVEL='1996' and T.MONTH_LEVEL in('5','6','7') and P.CLASS_LEVEL='FDXAQ1N5U026' and P.DIVISION_LEVEL='XRLXY6H61SLC' and C.RETAILER_LEVEL='RQJNEN0UPKMQ' and H.ALL_LEVEL ='DEFGHIJKLMNO' group by Customer_level, Product_level, Time_level, Channel_level

Algorithme Génétique

Un AG est un algorithme itératif de recherche d'optimum, il manipule une population de taille constante. Cette dernière est formée de candidats appelés individus.

Chaque individu représente le codage d'une solution potentielle au problème à résoudre. Il est constitué d'un ensemble d'éléments appelés gènes, pouvant prendre plusieurs valeurs appartenant à un alphabet non forcément numérique]. A chaque itération (génération) est créée une nouvelle population avec le même nombre d'individus. Cette génération consiste en des individus mieux "adaptés" à leur environnement tel qu'il est représenté par la fonction sélective. Au fur et à mesure des générations, les individus vont tendre vers l'optimum de la fonction sélective. La création d'une nouvelle population, à partir de la précédente, se fait par application des opérateurs génétiques que sont : la sélection, le croisement et la mutation. Ces opérateurs sont stochastiques.

La sélection des meilleurs individus est la première opération dans un algorithme génétique. Au cours de cette opération, l'algorithme sélectionne les éléments pertinents qui optimisent le mieux la fonction.

Le croisement permet de générer deux individus nouveaux "enfants" à partir de deux individus sélectionnés "parents"

La mutation réalise l'inversion d'un ou plusieurs gènes d'un individu.

Les AGs fonctionnent sur une population d'individus N (la population représente tous les schémas de fragmentation possibles), que l'on croise entre eux pour trouver un individu "parfait" qui correspond au schéma de fragmentation optimal de l'entrepôt de données.

L'algorithme génétique appliqué par Boukhalfa [BBK06] suit la démarche suivante:

Génération aléatoire de la population initiale

Calcul de la fonction sélective

Répéter

 Sélection

 Croisement

 Mutation

 Calcul de la fonction sélective

Jusqu'à satisfaction du critère d'arrêt

Processus de création d'un individu

Avant de décrire le processus de codage des individus de la population, il est réalisé :

1. l'extraction de tous les prédicats de sélection simples utilisés par les n requêtes.
2. L'attribution à chaque table de dimension $D_i (1 \leq i \leq d)$ d'un ensemble de prédicats simples EP_{SDi} qui lui est propre.
3. Toute table de dimension D_i ayant $EP_{SDi} = \square$, ne sera pas fragmentée. Soit $D_{candidat}$ l'ensemble de toutes les tables de dimension ayant leur ensemble de prédicats simples non vide. Soit g la cardinalité de l'ensemble $D_{candidat}$ ($g \leq d$).
4. L'application de l'algorithme COM-MIN[OMT99] à chaque ensemble de prédicats simples d'une table de dimension D_i . Il fournit en sortie une forme complète et minimale de ces ensembles. La règle de complétude et de minimalité assure que si une table est fragmentée en au moins deux fragments, elle sera accédée différemment par au moins une application [OMT99].

Une représentation des fragments horizontaux

L'analyse des clauses représentant les fragments horizontaux permet d'effectuer une partition du domaine des attributs de fragmentation en sous domaines appelés sous domaines stables (SDS) [Simonet et Simonet, 1994]. Les éléments de cette partition sont déterminés par les prédicats de simples.

Chaque individu (un schéma de fragmentation) est donc représenté par un tableau d'entier ou par un cube.

A partir de cette table, on peut déduire si la fragmentation de l'entrepôt se fera ou non sur l'attribut concerné et cela en vérifiant tous les sous domaines de l'attribut si ils ont la même valeur ou non.

Le codage du génome doit satisfaire les règles de correction. Pour cela, l'individu est représenté par un tableau d'entier pour chaque attribut, où une cellule correspondrait à un sous domaine de l'attribut.

Intérêt de ce codage

Le codage présente plusieurs avantages :

1. les nouveaux individus issus du croisement sont compris dans la limite du domaine.
2. Il n'y a pas de chevauchement de fragments.
3. Tous les sous domaines sont représentés.
4. La mutation ne peut pas rendre un individu invalide

Cette représentation d'un individu permet de définir les fragments d'une table des faits ou bien d'une table dimension.

L'avantage d'une telle représentation est que, quelle que soit la façon dont est obtenu l'individu, elle est toujours valide dans les cas d'une génération aléatoire ou un croisement. Eviter de vérifier à chaque création d'un individu s'il est valide ou non représente un gain de temps en calcul.

Sélection des individus

La méthode utilisée est la méthode de la roulette pour sélectionner les individus parents.

Dans cette méthode chaque individu est accompagné de son évaluation. Plus cette évaluation est élevée, plus l'individu a de chances d'être sélectionné.

– Une fois les individus choisis, on détermine si on va les croiser ou non et cela en tirant un chiffre aléatoire;

–Si ce chiffre est supérieur à un certain taux de croisement (en général fixé entre 70 et 90 %), on ne croise pas et les individus sont réinjectés directement dans la population de génération suivante.

–Si le père et la mère sont trop semblables, un seul sera réinjecté.

Dans le cas contraire on crée deux enfants à partir des deux parents. On ajoute alors ces enfants à la population.

Types de croisements

Le croisement utilisé est un croisement multipoints car les prédicats sont placés les uns à la suite des autres dans le vecteur d'entiers quand l'individu est créé. Les individus sont croisés une fois sur chaque prédicat, pour ne pas désavantager le croisement d'un prédicat par rapport à un autre.

Chaque prédicat dispose d'un nombre d'entiers différents. Si on n'effectue qu'un croisement par individu le prédicat qui a un grand nombre d'entiers (ex : ville : autant d'entiers que de villes) aura une probabilité de croisement supérieure à celui qui n'a que quelques entiers (sexe : masculin féminin donne 2 entiers).

Cette opération de sélection est effectuée tant que la population n'a pas atteint le nombre convenable d'individus.

Évaluation de l'individu (la fonction sélective)

L'évaluation d'un individu permet de définir que l'individu est meilleur par rapport à un autre. Cette évaluation donne un pourcentage qui est la somme de deux paramètres:

1- respect du seuil : par défaut 55 points sur cent sont alloués. Si le nombre de fragments obtenus est égal à plus ou moins cinq pourcent le nombre de fragments préconisé par l'administrateur (ou seuil), tous les points sont alloués. Par la suite plus on s'éloigne du seuil, moins de points sont ajoutés au résultat.

2- rapidité des requêtes : un nombre de points uniforme est alloué à chaque requête.

Par défaut, comme il reste 45 points (100 - 55 du seuil) et que nous avons 15 requêtes, 3 points sont alloués à chaque requête. La rapidité d'une requête dépend d'un calcul selon un modèle de coût qui nous donne le nombre d'entrées sorties qu'effectue la requête. Ce modèle est exprimé par l'équation suivante [BBK06]:

$$C_j = \frac{M_j \cdot F_j \cdot L_j}{PS_j}$$

Où, M_j , F , L et PS représentent le nombre de prédicats de sélection utilisés pour définir les fragments de sous schéma en étoile S_j , la cardinalité de la table des faits (nombre de tuples) F , la longueur, en octets, de chaque tuple de la table des faits et la taille d'une page disque (en octets), respectivement. Le coût total pour exécuter l'ensemble des requêtes est exprimé par l'équation suivante:

$$C_{total} = \sum_{j=1}^n C_j$$

Le problème de fragmentation d'un entrepôt de données relationnel consiste à partitionner un schéma en étoile, tel que le coût total d'exécution de requêtes soit minimal ou en d'autres termes, le gain soit maximisé :

Maximiser $Eval(Q, FS_i) = (TIOC(Q, \phi) - TIOC(Q, FS_i))$ tel que $N_i \leq W$, avec $TIOC(Q, \phi)$ représente le coût total d'exécution de l'ensemble de requêtes sur le schéma de l'entrepôt non fragmenté.

La mutation

Dans la nature, les mutations sont fréquentes et entraînent des variations plus ou moins marquées de l'individu. Dans ce modèle le taux de mutation choisi se situe entre 30% et 6%, c'est le taux habituellement utilisé. Il faut choisir un juste milieu entre une trop faible et une trop forte mutation. Les mutations s'effectuent au niveau des attributs de l'individu. Suivant un nombre aléatoire, la mutation se fait sur un ou plusieurs attributs choisis de l'individu.

Bibliographie

- [APE88] P.G.M. Apers. « Data allocation in distributed database systems. *ACM transactions on database systems*, 13(3):263-304, 1988..
- [ARG96] R. Agrawal, A. Gupta, et S. Sarawagi.: *Modeling multidimensional databases. Rapport de recherche : IBM Almaden, centre de recherche, CA,1996.*
- [BAC95] Bäck T.,. *Evolutionary algorithms in theory and practice*, Oxford University Press, New York, 1995
- [BAE05] Baer H., *Partitioning in Oracle Database 10g Release 2, An Oracle White Paper May, 2005.*
- [BBA06] Bellatreche L., Boukhalfa K., Abdalla H. I., « SAGA : A Combination of Genetic and Simulated Annealing Algorithms for Physical Data Warehouse Design », in *23rd British National Conference on Databases*, July, 2006.
- [BBK05] Bellatreche L., Boukhalfa K., « An Evolutionary Approach to Schema Partitioning Selection in a Data Warehouse Environment », *Proceeding of the International Conference on Data Warehousing and Knowledge Discovery (DAWAK'2005)*, vol. , p. 115-125., August, 2005
- [BBK06] K Boukhalfa , L. Bellatreche., « Sélection de schéma de fragmentation horizontale dans les entrepôts de données :formalisation et algorithmes » *Revue d'Ingénierie des Systèmes d'Information (ISI)* , vol. 11 (6/2006), , pp. 55-82 Novembre, 2006
- [BEL00] L.Bellatreche « Utilisation des Vues Matérialisées, des Index et de la Fragmentation dans la Conception Logique et Physique d'un Entrepôt de Données » thèse de doctorat, université Clermont-Ferrand II Décembre 2000.
- [BEL98] L. Bellatreche, K. Karlapalem, et Q. Li.: *Derived horizontal class partitioning in oodbss : Design strategy, analytical model and evaluation. In the 17th International Conference on the Entity Relationship Approach*, 1998.
- [BKD98] R. G. Bello, K. Dias, A. Downing, Feenan Jr. J., W. D. Norcott, H. Sun, A. Witkowski, et M. Ziauddin: *Materialized views in oracle. Proceedings of the International Conference on Very Large Databases*, 1998.
- [BKA00] Bellatreche L., Karlapalem K., A. S., « Algorithms and Support for Horizontal Class Partitio- ning in Object-Oriented Databases », in the *Distributed and Parallel Databases Journal*, vol. 8, n° 2, p. 155-179., April, 2000.
- [BKL98a] L.Bellatreche K.Karlapalem and Q.Li *Complex methods and class allocation in distributed object-oriented databases. In the 5th international conference on Object Oriented Information System(OOIS'98). September 1998.*
- [BKL98b] L.Bellatreche K.Karlapalem and Q.Li *Decrived horizontal class partitioning in*

- oodbss: Design strategy, analytical model and evaluation. In the 17th international conference on the Entity Relationship Approach (ER'98), P 465-479 November 1998.*
- [BKM00] Bellatreche L., Karlapalem K., Mohania M « What can Partitioning do for your Data Warehouses and Data Marts », *Proceedings of the International Database Engineering and Application Symposium (IDEAS'2000)*, vol. , p. 437-445., September, 2000.
- [BLA86] J. A. Blakeley, P. Larson, et F. W. Tompa : Efficiently updating materialized views. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1986.
- [BME72] Bayer R., McCreight E.M., *Organization and Maintenance of Large Ordered Indices* , Acta Informatica , 1 :173189. 1972.
- [BMT02] Body, M., Miquel, M., Bédard, Y., Tchounikine, A.: A multidimensional and multiversion structure for OLAP applications. *DOLAP 2002, ACM Fifth International Workshop on Data Warehousing and OLAP*, 2002.
- [BRI00] Briard B. MAUD : une méthode pour auditer la qualité des données. *Mémoire de DRT SIO*, 2000.
- [BSL04] Bellatreche L., Schneider M., Lorinquer H., Mohania M « Bringing Together Partitioning, Materialized Views and Indexes to Optimize Performance of Relational Data Warehouses », *Proceeding of the International Conference on Data Warehousing and Knowledge Discovery (DAWAK'2004)*, vol. , p. 15-25., September, 2004.
- [CHA82] J.Chang.. A heuristic approach to distributed query processing. *Proceedings of the International Conference on Very Large Databases*, p 54-61 September 1982.
- [CHA97] S. Chaudhuri et U. Dayal: An overview of data warehousing and olap technology. *Sigmod Record*, 1997.
- [CHA98] S. Chaudhuri et V. Narasayya: Autoadmin 'what-if' index analysis utility. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1998.
- [CHA99] S. Chaudhuri et V. Narasayya: Index merging. *Proceedings of the International Conference on Data Engineering (ICDE)*, 1999.
- [CHN97] Chaudhuri S., Narasayya V., « An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server », *Proceedings of the International Conference on Very Large Databases*, vol. , p. 146-155., August, 1997
- [CHY99] Chee-Yong C., *Indexing Techniques in Decision Support Systems*, Ph.d. thesis, University of Wisconsin - Madison., 1999

-
- [COU98] Council O., « APB-1 OLAP Benchmark, Release II », 1998.
- [CRS01] Cauvet, C., Rosenthal-Sabroux, C. : *Ingénierie des systèmes d'information sous la direction de Corine Cauvet, Camille Rosenthal-Sabroux*. Paris Hermès Science Publications, France, 2001.
- [CVN00] S.Chaudhuri and V. Narasayya.. Automating statistics management for query optimizers. *Proceeding of the International conference on Data Enginneering(ICDE)*, p 339-348 March 2000.
- [DAT99] A. Datta, K. Ramamritham, et H. Thomas. Curio: A novel solution for efficient storage and indexing in data warehouses. *Proceedings of the International Conference on Very Large Databases*, 1999.
- [DBA88] Devlin, B. A., Murphy, P. T.: *Architecture for a Business and Information System*. IBM Systems Journal 27(1), 1988.
- [DJB06] Darmont,J.,&Boussaïd,O.(Eds.). «Managing and Processing Complex Data for Decision Support». Idea Group Publishing. 2006
- [DRT99] Datta A., Ramamritham K., Thomas H., « Curio : A Novel Solution for Efficient Storage and Indexing in Data Warehouses », *Proceedings of the International Conference on Very Large Databases*, vol. , p. 730-733., September, 1999
- [DSH98] Dinter, B., Sapia, C., Höfling, G., Blaschka, M.: *The OLAP market: state of the art and research issues*. DOLAP 1998, ACM First International Workshop on Data Warehousing and OLAP, 1998.
- [ECB95] Ezeife, C. I., & Barker, K.. «A Comprehensive Approach to Horizontal Class Fragmentation in a Distributed Object System». *Distributed and Parallel Data-bases*, 3(3), 247–272. 1995
- [FKL97] C.W. Fung, K. Karlapalem,and Q. Li.. Cost-driven evaluation of vertical class partitioning in object oriented databases. In *Fifth International Conference On Database Systems For Advanced Applications (DASFAA'97)*, Melbourne, Australia, p 11-20 April 1997.
- [FR97b] Franco J.-M. : *Le Data Wharehouse : objectifs, définitions, architectures*, Edition Eyrolles, 1997.
- [FRA98] Jean-François Goglin : *La construction du datawarehouse. Du datamart au dataweb*, Nouvelles Technologies Informatiques; Ed. Hermes, 1998.
- [GAL97] Fred Glover and Manuel Laguna,. "Tabu Search.", Kluwer Academic Publishers, page 50, 72, 110. 1997
- [GMR98] Golfarelli, M., Maio, D., Rizzi, S.: *The Dimensional Fact Model: A Conceptual Model for Data Warehouses*. *International Journal of Cooperative Information Systems*, 1998.

-
- [GMR99] Golfarelli M., Maio D., Rizzi S., « Vertical Fragmentation of Views in Relational Data Ware- houses », *Proceedings of Sistemi Evoluti per Basi di Dati*, vol. , p. 19-33,. 1999
 - [GRU96] J.R Gruser. « Modèle de coût pour l'optimisation de requête objet ». Thèse de doctorat, Université de Paris VI. Décembre 1996.
 - [GUP99] Gupta H Selection and Maintenance of Views in a Data Warehouse, Ph.d. thesis, Stanford University., September, 1999.
 - [GWW96] Guo S., Wei S., Weiss M. A« On Satisfiability, Equivalence, and Implication Problems In- volving Conjunctive Queries in Database Systems », *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, n° 4, p. 604-612., August,.1996.
 - [HMV99] Hurtado, C. A., Mendelzon, A. O., Vaisman, A. A.: Updating OLAP dimensions. DOLAP 1999, ACM Second International Workshop on Data Warehousing and OLAP, 1999.
 - [HNS95] P.J Hass and A.N Swami.. Sampling-based selectivity estimation for joins using augmented frequent value statistics. *Proceeding of the International Conference on Data Engineering (ICDE)*, p 522-531,1995.
 - [HYP] Hyperion. Hyperion Essbase OLAP Server. [http ://www.hyperion.com/](http://www.hyperion.com/).
 - [INF97] Informix Corporation: Informix-online extended parallel server and informix-universal server: A new generation of decision-support indexing for enterprise data warehouses. White Paper, 1997.
 - [INM97] Inmon W.-H., Zachman John A., Geiger Jonathan G. : Data Stores, Entrepôts and the Zachman Framework, 1997 .
 - [IOA93] Y.E. Ioannidis.. University of serial histograms. *proceeding of the International conference Very Large Databases*, p 256-267, August 1993.
 - [IOK90] Ioannidis Y., Kang Y., « Randomized algorithms algorithms for optimizing large join queries », *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. , p. 9-22,. 1990
 - [IWH94] Inmon, W.H., Hackarton, R.D.: Using the Data Warehouse. John Wiley & Sons, 1994.
 - [JAG99] H. Jagadish, L. V. S. Lakshmanan, et D. Srivastava: What can hierarchies do for your data warehouses *Proceedings of the International Conference on Very Large Databases*, 1999.
 - [JAL96] Brigitte Jaumard, Mihnea Stan, and Jacques Desrosiers., , "Tabu search and a quadratic relaxation for the satisfiability problem. In Cliques, Coloring, and Satisfiability" :Second DIMACS Implementation Challenge, volume 26 of DIMACS Series in Discrete Mathematics and Theoretical Computer Science,

- [KGV83] Kirkpatrick S., Gelatt C. D., Vecchi M. P., « Optimization by Simulated Annealing », *Science*, vol. 220, n° 4598, p. 671-680, May, 1983
- [KIM96] Kimball R.: *The entrepôt toolkit*, John Wiley and Sons, 1996.
- [KIM97] R. Kimball: *A dimensional modeling manifesto*. DBMS Magazine, 1997.
- [KOT99] Y. Kotidis et N. Roussopoulos: *Dynamat: A dynamic view management system for data warehouses*. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1999.
- [LAQ97] W.Labio D.Quass and R.Adelberg. *Physical database design for data warhouses*. *Proceeding of the International Conference on Data Engineering(ICDE)* ,1997.
- [LAR05] *Thèse de doctorat, Ecole Doctorale d'Angers Frédéric LARDEUX Approches hybrides pour les problèmes de satisfiabilité (SAT et MAX-SAT) 25 Novembre 2005.*
- [LHE95] Lesca H., Lesca E. : *Gestion de l'information : qualité de l'information et performances de l'entreprise*, Editions Litec, 1995.
- [LRA98] Lei H., Ross K. A., « Faster Joins, Self-Joins and Multi-Way Joins Using Join Indices », *Data and Knowledge Engineering*, vol. 28, n° 3, p. 277-298 November, 1998.
- [MAL97] Mazure et al., Bertrand Mazure, Lakhdar Sais, and Eric Grégoire., "Tabu search for SAT". In *Proc. Of the AAAI-97/IAAI-97*, Providence, Rhode Island pages 281–285. 1997.
- [MEN97] A. Mendelzon. *OLAP: Concepts and products*.Talk at University of Toronto, 1997.
- [MHP99] McFadden, F. R., Hoffer, J. A., Prescott, M. B.: *Modern Database Management Fifth Edition*. Addison-Wesley, États-Unis, 1999.
- [MIC98] *MicroStrategy: The case for relational Olap*. Rapport technique, papier blanc, 1998.
- [MWM98] Munneke, D., Wahlstrom, K., & Mohania, M. K.. «Fragmentation of Multidimensional Databases». *10th Australasian Database Conference (ADC99)*, Auck- land, New Zealand (pp. 153–164). 1999.
- [NAA99] H.Naacke. *Modèle de coût pour médiateurs de bases de données hétérogènes*. *Thèse de doctorat, université de Versailles saint Quentin- en Yvelines.*, Septembre 1999.
- [NBK99] Noaman A. Y., Barker K., « A Horizontal Fragmentation Algorithm for the

- Fact Relation in a Distributed Data Warehouse », in the 8th International Conference on Information and Knowledge Management (CIKM'99), vol. , p. 154-161, November, 1999.*
- [NKR95] Navathe, S. B., Karlapalem, K., & Ra, M.. «A Mixed Fragmentation Methodology for Initial Distributed Database Design. » *Journal of Computer and Software Engineering*, 3(4). 1995.
- [NYS97] Ng, Y.-K., & Seetharaman, A.. *A Query-Based Horizontal Fragmentation Approach for Disjunctive Deductive Databases. DEXA Workshop* (pp. 604–609). 1997
- [OMT99] Özsu M. T., Valduriez P. *Principles of Distributed Database Systems : Second Edition*, Prentice Hall., 1999.
- [ONB97] O'Neil, Bonnie: *Oracle Data warehousing unleashed* Bonnie O'Neil...[et al.]. Sams Indianapolis, Ind., Etats-Unis, 1997.
- [ONE97] P. O'Neil et D. Quass: *Improved query performance with variant indexes. In Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1997.
- [OPQ97] O'Neil P., Quass D., « *Improved Query Performance with Variant Indexes* », *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. , p. 38-49 May, 1997.
- [OQ97] O'Neil P., Quass D., *Improved Query Performance with Variant Indexes* , in *ACM SIGMOD International Conference on Management of Data (SIGMOD 1997)*, Tucson, USA, pp. 3849. 1997.
- [OZS99] M. T. Ozsu et P. Valduriez: *Principles of Distributed Database Systems. Second Edition*. Prentice Hall, 1999.
- [PKN91] Pernul, G., Karlapalem, K., & Navathe, S. B.. «*Relational Database Organization Based on Views and Fragments*». *International Conference Database and Expert Systems Applications (DEXA 91)*, Berlin, Germany (pp. 380{386). Springer-Verlag. 1991.
- [PSA04] Papadomanolakis S., Ailamaki A., « *AutoPart : Automating Schema Design for Large Scientific Databases Using Data Partitioning* », *Proceedings of the 16th International Conference on Scientific and Statistical Database Management (SSDBM 2004)*, vol. , p. 383-392 ,June 2004.
- [RAM98] R. Ramakrishnan.. *Database management systems*. WCB /McGraw Hill, 1998.
- [RBN02] Rifaieh, R., Benharkat, N. A.: *Query-based data warehousing tool. DOLAP 2002, ACM 5th International Workshop on Data Warehousing & OLAP*, 2002.
- [RED97] *Red Brick Systems: Star schema processing for complex queries. Papier Blanc*, 1997.

-
- [RFZ95] Ravat, F., & Zurfluh, G.. «Issues in the Fragmentation of Object-Oriented Databases»«. 2nd Basque International Workshop on Information Technology (BIWIT 95), San Sebastian, Spain (pp. 150–160). 1995
- [RZL02] Rao J., Zhang C., Lohman G., Megiddo N., « Automating Physical Database design in a parallel database », *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. , p. 558-569, June, 2002.
- [SAM94] Simonet A., Simonet M« Objects with Views and Constraints : From Databases to Knowledge Bases », in the *Proceeding of the International Conference on Object Oriented Information Systems, OOIS'94*, vol. , p. 182-195., December, 1994.
- [SCB03] Scalzo, Bert. : *Oracle DBA Guide to Data Warehousing and Star Schemas*. Prentice Hall, États-Unis, 2003.
- [SCN01] SCN Education B.V.: *Data warehousing the ultimate guide to building corporate business intelligence*. Braunschweig/Wiesbaden Vieweg. Grèce, 2001.
- [SKS99] Silberschatz, A., Korth, H. F., Sudarshan, S.: *Database System Concepts Third Edition*. WCB McGraw-Hill, États-Unis, 1999.
- [SMR00] Stöhr T., Märtens H., Rahm E« Multi-Dimensional Database Allocation for Parallel Data Warehouses », *Proceedings of the International Conference on Very Large Databases*, vol. , p. 273-284., 2000.
- [SNY04] Sanjay A., Narasayya V. R., Yang B « Integrating Vertical and Horizontal Partitioning Into Automated Physical Database Design », *Proceedings of the ACM SIGMOD International Conference on Management of Data*, vol. , p. 359., -370. June, 2004.
- [SRI96] D. Srivastava, S. Dar, H. Jagadish, and A. Y. Levy: *Answering queries with aggregation using views*. *Proceedings of the International Conference on Very Large Databases*, 1996.
- [SYB05] Sybase, *Sybase Adaptive Server Enterprise 15 Data Partitioning*, White paper., 2005.
- [TDB00] Theodoratos, D., Bouzeghoub, M.: *A general framework for the view selection problem for entrepôt design and evolution*. DOLAP 00, *ACM Third International Workshop on Data Warehousing and OLAP*, 2000.
- [THE98] Theodoratos D., Sellis T., "Datawarehouse Schema and Instance Design", *ER'9. 17th International Conference on Conceptual Modeling* 1998.
- [THG91] Tardieu H., Guthmann B.: *Le Triangle stratégique*. Les Editions d'Organisation, 1991.

-
- [ULL89] J.D. Ullman «Principles of database and Knowledge-Base Systems. vol . II. Computer Science Press, 1989.
- [ULL96] J. Ullman: Efficient implementation of data cubes via materialized views.Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD'96), 1996.
- [VAL87] P. Valduriez: Join indices. ACM Transactions on Database Systems, 1987.
- [VSS02] Vassiliadis, P., Simistsis, A., Skiadopoulos, S.: Conceptual modeling for ETL processes. National Technical University of Athens, Athens, Greece , 2002 .
- [WLD96] Wang Y., Liu J. C., Du, Hsieh J., « Video file allocation over disk arrays for video-on- demand », in Proceedings of the International Conference on Multimedia Computing and Systems(ICMCS'96), vol. , p. 160-173, June, 1996.
- [WMB97] Wu M.-C., Buchmann A., « Research Issues in data warehousing », in Datenbanksysteme in Büro, Technik und Wissenschaft(BTW'97), vol. , p. 61-82., March, 1997.
- [WMT05] Wehrle P., Miquel M., Tchounikine A., « A Model for Distributing and Querying a Data Ware- house on a Computing Grid », 11th International Conference on Parallel and Distributed Systems (ICPADS 2005), vol. , p. 203-209., July, 2005.
- [WUA97] M-C. Wu et A. Buchmann: Research issues in data warehousing. In Datenbanksysteme in Bro Technik und Wissenschaft (BTW'9 7), 1997.
- [YCC84] C. T. Yu and Chang C.C.. Distributed query processing. ACM computing Surveys,; p 399-433, December 1984.
- [YCG04] Yu J. X., Choi C.-H., Gou G., « Materialized View Selection as Constrained Evolution Opti- mization », IEEE Transactions On Systems, Man, and Cybernetics, Part 3, vol. 33, n° 4, p. 458-467., November, 2004.
- [YIC91] Y.E. Ioannidis and S. Christodoulakis.. On the propagation of errors in the size of join results. proceeding of the ACM SIGMOD International conference on management of data, p 268-277,June 1991.
- [YKL97] J. Yang, K. Karlapalem, and Q. Li.. Algorithms for materialized view design in data warehousing environment. Proceedings of International Conference on Very Large Databases, p 136-145 ,August 1997.
- [ZBA06] Ziyati E., Bellatreche L., Aboutajdine D., « Un algorithme génétique pour la sélection d'un schéma de fragmentation mixte dans les entrepôts de données », Ateliers des Systèmes décisionnels (ASD06),. December, 2006

Bibliographie

-
- [ZRL04] Zilio D. C., Rao J., Lightstone S., Lohman G. M., Storm A., Garcia-Arellano C., Fadden S., « DB2 Design Advisor : Integrated Automatic Physical Database Design », *Proceedings of the International Conference on Very Large Databases*, vol. , p. 1087-1097, August, 2004.
- [ZY094] Zhang, Y., & Orlowska, O.«On fragmentation approaches for distributed database design» *Information Sciences*, 1(3), 117–132. 1994